

Catalog of Metamorphic Relations For Security Testing

Authors:

Nazanin Bayati Chaleshtari, n.bayati@uottawa.ca

Fabrizio Pastore, fabrizio.pastore@uni.lu

Date	Author	Description
04/08/2022	Nazanin Bayati	First version of the catalog

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp{

/**
 * CWE Definition:
 *
 * CWE_138:
 * The software receives input from an upstream component, but it does not neutralize
 * or incorrectly neutralizes special elements that could be interpreted as control
 * elements or syntactic markers when they are sent to a downstream component.
 *
 * CWE_150 (Child of CWE_138):
 * The software receives input from an upstream component, but it does not neutralize
 * or incorrectly neutralizes special elements that could be interpreted as escape,
 * meta, or control character sequences when they are sent to a downstream component.
 *
 * CWE_551:
 * If a web server does not fully parse requested URLs before it examines them for
 * authorization, it may be possible for an attacker to bypass authorization
 * protection.
 *
 * CWE_552:
 * The product makes files or directories accessible to unauthorized actors, even
 * though they should not be.
 *
 * CWE_668:
 * The product exposes a resource to the wrong control sphere, providing unintended
 * actors with inappropriate access to the resource.
 *
 * CWE_707:
 * The product does not ensure or incorrectly ensures that structured messages or
 * data are well-formed and that certain security properties are met before being
 * read from an upstream component or sent to a downstream component.
 *
 * CWE_921:
 * The software stores sensitive information in a file system or device that does not
 * have built-in access control.
 *
 * Idea:
 * A file path passed in the URL of a request should never enable a
 * user to access data that is not provided by the user interface.
 *
 *
 * Comments:
 * (1) Defines the separator variable
 * (2) For loop iterates for 3 times to cover multiple occurrence of the path
 *     separator as well.
 * (3) Second for loop iterates over all actions of the Input(1).
 * (4) Stores the parameters of the current action in a variable.
 * (5) Defines the new URL.
 * (6) Verifies whether the current action has been not performed by an
 *     administrator.
 * (7) Checks if the current action has been performed after a login.

```

```

* (8) Defines a follow-up input that is a copy of the source input.
* (9) Adds to the end of the current URL a relative path to a file.
* (10) Verifies that the given path was not tried in a previous execution of the
*      loop (to speed up).
* (11) Verifies that the follow-up input does not lead to a file
*      Or the returned file is something that the user has the right to access.
*      Or the system shows an error page.
*
*/

```

```

MR CWE_138_150_551_552_668_707_921_OTG_AUTHZ_001b {
{
    var sep="/"; // (1)
    for ( var par=0; par < 3; par++ ){ // (2)

        for ( Action action : Input(1).actions() ){ // (3)
            var pos = action.getPosition(); // (4)
            var newUrl = action.urlPath+sep+RandomFilePath(); // (5)
            IMPLIES(
                !isAdmin(action.user) && // (6)
                afterLogin(action) && // (7)
                CREATE( Input(2), Input(1) ) && // (8)
                Input(2).actions().get(pos).setUrl( newUrl ) && // (9)
                notTried( action.getUser(), newUrl ) // (10)
            ,
            TRUE ( // (11)
                Output(Input(2),pos).noFile() ||
                userCanRetrieveContent( action.getUser(), Output(Input(2),pos).file() ) ||
                Output(Input(2),pos).isError()
            ); //end-IMPLIES
        } //end-for
        sep=sep+"../";
    } //end-for
}
} //end-MR
} //end-package

```



```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp{

/**
 * CWE Definition:
 *
 * CWE_15:
 * One or more system settings or configuration elements can be externally controlled
 * by a user.
 *
 * CWE_639: The system's authorization functionality does not prevent one user from
 * gaining access to another user's data or record by modifying the key value
 * identifying the data.
 *
 * Idea:
 * To check the system's authorization functionality we will check whether the user
 * can access the data from another user if we directly send the parameter value in
 * the URL or not.
 * We are going to change the value of the all parameters of the actions that the
 * user can take. To change the parameter value, we replace it with the value from
 * another user, and it is passed to URL. By randomly changing the parameter values
 * passed to URL, a user should not be able to retrieve content that she cannot
 * retrieve from GUI.
 *
 *
 * (1) The first loop iterates over all the actions of the input sequence.
 * (2) The second iterates over all the parameters of the action.
 * (3) The third loop iterates over all used values by other users.
 * (4) Stores the parameters of the current action in a variable.
 * (5) Defines the follow-up input.
 * (6) Sets a parameter value to a value that is used by other users.
 * (7) Checks if the content of the output is either an error message
 * Or some content that can be retrieved from the GUI.
 */

MR CWE_15_639_OTG_AUTHZ_004 {
{
for ( Action action : Input(1).actions() ){ // (1)
    for ( var par=0; par < action.getParameters().size() && notTried( action.getUser(),
        action.url); par++) { // (2)

        for ( usedValue : parameterValuesUsedByOtherUsers(action, par) ){ // (3)
            var pos = action.getPosition(); // (4)

            IMPLIES(
                CREATE ( Input(2), Input(1) ) && // (5)
                Input(2).actions().get(pos).setParameterValue(par, usedValue) // (6)
            ,
            OR( // (7)
                Output(Input(2),pos).isError(),
                userCanRetrieveContent( action.user, Output(Input(2),pos) ))
            ); // end-IMPLIES
        } // end-for
    }
}
}

```

```
    }//end-for  
  }//end-for  
}  
} //end-MR  
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp{

/**
 * CWE Definition:
 * CWE_20:
 * The product receives input or data, but it does not validate or incorrectly
 * validates that the input has the properties that are required to process the data
 * safely and correctly.
 *
 * CWE_73:
 * The software allows user input to control or influence paths or file names that
 * are used in filesystem operations.
 *
 * CWE_99:
 * The software receives input from an upstream component, but it does not restrict
 * or incorrectly restricts the input before it is used as an identifier for
 * a resource that may be outside the intended sphere of control.
 *
 * CWE_219:
 * The application stores sensitive data under the web document root with
 * insufficient access control, which might make it accessible to untrusted parties.
 *
 * CWE_220:
 * The application stores sensitive data under the FTP server root with insufficient
 * access control, which might make it accessible to untrusted parties.
 *
 * CWE_528;
 * The product generates a core dump file in a directory, archive, or other resource
 * that is stored, transferred, or otherwise made accessible to unauthorized actors.
 *
 * CWE_530:
 * A backup file is stored in a directory or archive that is made accessible to
 * unauthorized actors.
 *
 * CWE_642:
 * The software stores security-critical state information about its users, or the
 * software itself, in a location that is accessible to unauthorized actors.
 *
 * CWE_732:
 * The product specifies permissions for a security-critical resource in a way that
 * allows that resource to be read or modified by unintended actors.
 *
 * Idea:
 * A file path passed in a parameter should never enable a
 * user to access data that is not provided by the user interface.
 *
 * (1) Iterates over the actions of the Input(1).
 * (2) The second for loop iterates over the parameters of the action.
 * (3) Stores the parameters of the current action in a variable.1
 * (4) To speed up the process, verifies that the URL has not been tried before.
 * (5) Creates the follow-up input, named Input(2).

```

```

* (6) Sets the value of a parameter to a random file path.
* (7) Verifies that the system shows an error page
* Or the returned content is something that the user has the right to access.
*
*/

MR CWE_20_73_99_219_220_528_530_642_732_OTG_AUTHZ_001a {
{
    for ( Action action : Input(1).actions() ){                                //(1)
        for ( var par=0; par < action.getParameters().size(); par++ ){          //(2)
            var pos = action.getPosition();                                       //(3)
            IMPLIES(
                notTried( Input(1).actions().get(pos).getUrl() ) &&                //(4)
                CREATE( Input(2), Input(1) ) &&                                  //(5)
                Input(2).actions().get(pos).setParameterValue(par, RandomFilePath()) //(6)
            ,
            OR(                                                                    //(7)
                isError( Output(Input(2),pos)) ,
                userCanRetrieveContent(action.getUser(), Output(Input(2),pos)) )

            );//end-IMPLIES
        }//end-for
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;

package smrl.mr.owasp {

/*
 * CWE Definition:
 * The product exposes sensitive information to an actor that is not explicitly
 * authorized to have access to that information.
 *
 * Idea:
 * Inadequate checks on the tags search fields can lead to an access level violation.
 *
 * Comments:
 * (1) The first loop iterates over all actions of Input(1).
 * (2) The second loop iterates over all actions of Input(2).
 * (3) The third for loop iterates over a list of tags, in which the same user is
 * taking actions that are performed after the login (need to be authorized),
 * and both Input(1) and Input(2) contain a list of tags.
 * (4) Stores the tag(i) from the list of tags of Input(1) in a variable.
 * (5) For loop iterates over the list of tags of Input(2).
 * (6) Verifies that the tags are the same.
 * (7) Creates a follow-up input with different credentials as Input(1), named
 * Input(3).
 * (8) Creates a follow-up input with different credentials as Input(2), named
 * Input(4).
 * (9) Verifies that an unauthorized user cannot perform the action or they (the
 * follow-up inputs) should have the same tags.
 */

MR CWE_200 {
{
    for ( action1 : Input(1).actions ){                                //(1)
        for ( action2 : Input(2).actions ){                            //(2)
            for ( var i=0;afterLogin(action1) && afterLogin(action2) &&
                action1.user.isSimilar(action2.user) &&
                Output(Input(1), action1.position).containListOfTags() &&
                Output(Input(2), action2.position).containListOfTags() &&
                i<Output(Input(1), action1.position).listsOfTags().size;i++)    //(3)
            {
                var listOfTags1 = Output(Input(1), action1.position).listsOfTags().get(i);    //(4)
                for ( listOfTags2 : Output(Input(2), action2.position).listsOfTags() ) {    //(5)
                    IMPLIES (
                        listOfTags1.equals( listOfTags2 ) &&                //(6)
                        CREATE ( Input(3), changeCredentials ( Input(1), User() )) &&    //(7)
                        CREATE ( Input(4), changeCredentials ( Input(2), User() ))    //(8)
                    ,
                        (isError(Output(Input(3),action1.position)) ||    //(9)
                        isError(Output(Input(4),action2.position)) ||
                        Output(Input(3),action1.position).listOfTags(listOfTags1.id).equals (
                        Output(Input(4),action2.position).listOfTags(listOfTags2.id)) ));
                } //end-For_Tag2
            } //end-For_tag1
        } // end-For_action2
    } //end-For_action1
}
}

```

```
}  
} //end_MR  
} // end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp{

/*
 * CWE Definition:
 * The software uses external input to construct a pathname that is intended to
 * identify a file or directory that is located underneath a restricted parent
 * directory, but the software does not properly neutralize special elements
 * within the pathname that can cause the pathname to resolve to a location that
 * is outside of the restricted directory.
 *
 * Idea:
 * A file path passed in a parameter should never enable a
 * user to access data that is not provided by the user interface.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our
 *     framework by clicking on OK.
 * (2) For loop iterates over all actions of the Input(1)
 * (3) Stores the parameters of the current action in a variable
 * (4) Iterates over the actions that contain a form input.
 * (5) Checks if the action submits the form; i.e. the action contains clicking on
 *     submit button.
 * (6) Verifies that the URL has not been tried before.
 * (7) Filters the dynamic content.
 * (8) Creates Input(2) as the follow-up input.
 * (9) Set the path to a random file as the form input value.
 * (10) The system should show a content that the user has right to access,
 *      Or it should return an empty file or show an error page to the user.
 */

MR CWE_22 {

{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        var pos = action.getPosition(); // (3)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (4)
            {
                IMPLIES(
                    action.isClickOnButton && // (5)
                    notTried( x+action.url, Input(1).actions().get(pos).getElementURL())&& // (6)
                    userCanRetrieveContent(action.getUser(), Output(Input(1),pos))&& // (7)
                    CREATE(Input(2), Input(1) ) && // (8)
                    Input(2).actions.get(pos).setFormInput(x, "../"+RandomPath()) // (9)
                ,
                OR( // (10)
                    userCanRetrieveContent(action.getUser(), Output(Input(2),pos))
                , OR(
                    Output(Input(2), pos).emptyFile,
                    isError(Output(Input(2), pos))))
            }
        }
    }
}
}

```

```
    );//end-IMPLIES
  }//end-for
} //end-for
}
} //end-MR
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import sun.security.util.Password

package smrl.mr.owasp {

/**
 * CWE Definition:
 * Using an empty string as a password is insecure.
 *
 * Idea:
 * Trying to give an empty string as the input value for the password to check the
 * system is vulnerable or not.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1)
 * (2) Stores the parameters of the current action in a variable
 * (3) Iterates over the actions that contain a form input.
 * (4) Checks if the action submits the form; i.e. the action contains clicking on
 * submit button.
 * (5) Filters the dynamic content.
 * (6) Verifies that the URL has not been tried before.
 * (7) Creates Input(2) as the follow-up input.
 * (8) Set an empty string as the password value.
 * (9) The system should show a content that the user has right to access
 * Or an error page or a different result to the user.
 */

MR CWE_258 {
{
    for ( Action action : Input(1).actions() ) {                                //(1)
        var pos = action.getPosition();                                           //(2)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++) //(3)
        {
            IMPLIES(
                action.isClickOnButton &&                                           //(4)
                userCanRetrieveContent(action.getUser(), Output(Input(1),pos)) &&    //(5)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL())&& //(6)
                CREATE ( Input(2), Input(1) )&&                                     //(7)
                Input(2).actions.get(action.position).setFormInputOfType(x, "", "password") //(8)
            ,
            OR(                                                                    //(9)
                isError(Output(Input(2), pos)),
                different ( Output(Input(1),pos), Output(Input(2),pos)))

            );//end-IMPLIES
        }//end-for
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp{

/**
 *
 * CWE Definition:
 * CWE_262:
 * If no mechanism is in place for managing password aging, users will have no
 * incentive to update passwords in a timely manner.
 *
 * CWE_263:
 * Allowing password aging to occur may result in the possibility of diminished
 * password integrity.
 *
 * CWE_309:
 * The use of password systems as the primary means of authentication may be subject
 * to several flaws or shortcomings, each reducing the effectiveness of the
 * mechanism.
 *
 * CWE_324:
 * The product uses a cryptographic key or password past its expiration date, which
 * diminishes its safety significantly by increasing the timing window for cracking
 * attacks against that key.
 *
 * Idea:
 * All above-mentioned CWE weaknesses are related to the password aging concept,
 * then, we merged them into one MR.
 * We try to find logout and login actions and change the time passed between these
 * two actions with the wait() function to check whether the system has any password
 * aging mechanism.
 *
 *
 * Comments:
 * (1) For loop iterates over the Input(1)'s actions.
 * (2) Checks if the current action x is doing "log in".
 * (3) Checks that the action-x of the Input(1) does not include any errors.
 * (4) Creates the follow-up Input by copying the Input(1), named Input(2)
 * (5) Add a wait action to Input(2) that moves time forward for a year i.e. the
 *     expected time for resetting the password (password aging).
 * (6) Based on the above description the results should be different; it shall
 *     prompt a password update request.
 *
 */

MR CWE_262_263_309_324 {
{
    for ( var x=0; x < Input(1).actions().size() ; x++ ){                //(1)
        IMPLIES (
            isLogin( Input(1).actions().get(x) ) &&                      //(2)
            !isError ( Output(Input(1),x+1) )&&                          //(3)
            CREATE ( Input(2) , Input(1) ) &&                            //(4)
            Input(2).addAction ( x, Wait( 60*60*24*30*12*1000) )          //(5)

```

```
        ,
        different ( Output(Input(2),x+2) ,  Output(Input(1),x+1))           //(6)
    ); //end-IMPLIES
} //end-for
}
} //end-MR
} //end-package
```

```

import static smrl.mr.language.Operations.*
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 * CWE Definition:
 * CWE_266:
 * A product incorrectly assigns a privilege to a particular actor, creating an unintended
 * sphere of control for that actor.
 *
 * CWE_267:
 * A particular privilege, role, capability, or right can be used to perform unsafe actions
 * that were not intended, even when it is assigned to the correct entity.
 *
 * CWE_268:
 * Two distinct privileges, roles, capabilities, or rights can be combined in a way that allows
 * an entity to perform unsafe actions that would not be allowed without that combination.
 *
 * CWE_269:
 * The software does not properly assign, modify, track, or check privileges for an actor,
 * creating an unintended sphere of control for that actor.
 *
 * CWE_285:
 * The software does not perform or incorrectly perform an authorization check when an actor
 * attempts to access a resource or perform an action.
 *
 * CWE_522:
 * The product transmits or stores authentication credentials, but it uses an insecure method
 * that is susceptible to unauthorized interception and/or retrieval.
 *
 * CWE_529:
 * The product stores access control list files in a directory or other container that is
 * accessible to actors outside of the intended control sphere.
 *
 * CWE_862:
 * The software does not perform an authorization check when an actor attempts to access a
 * resource or perform an action.
 *
 * CWE_863:
 * The software performs an authorization check when an actor attempts to access a resource or
 * perform an action, but it does not correctly perform the check. This allows attackers to
 * bypass intended access restrictions.
 *
 * Idea:
 * A URL that cannot be reached by a user while navigating the user interface should not
 * be available to that same user even when she directly requests the URL to the server.
 * For this reason, an input sequence that is valid for a given user, should not lead to
 * the same output when it is executed by another user, if it includes access to a URL with
 * these characteristics.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1)

```

```

* (2) Checks whether the user in User() is not a supervisor of the user performing the current
* action.
* (3) Verifies that the user cannot retrieve the URL of the action through the GUI (based on
* the data collected by the crawler).
* (4) Defines a follow-up input that matches the source input except that the credentials of
* User() are used in this case.
* (5) Verifies that the follow-up input leads to an error page
* Or the output generated by the action containing the URL indicated above leads to two
* different outputs in the two cases.
*
*/

```

```

MR CWE_266_267_268_269_285_522_529_862_863_OTG_AUTHZ_002 {
{
  for ( Action action : Input(1).actions() ){                                //(1)

    IMPLIES(
      !isSupervisorOf(User(), action.user) &&                                //(2)
      cannotReachThroughGUI( User(), action.url )&&                          //(3)
      CREATE( Input(2), changeCredentials(Input(1), User()) )                //(4)
    ,
    OR(                                                                           //(5)
      isError(Output(Input(1),action.position)),
      NOT( Output(Input(1),action.position).equals(Output(Input(2),action.position))
    )
  )); //end-IMPLIES
} //end-for
}} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_276: During installation, installed file permissions are set to allow anyone to modify
 * those files.
 *
 * CWE_277: A product defines a set of insecure permissions that are inherited by objects that
 * are created by the program.
 *
 * Idea:
 * We merged these two CWE weaknesses as they both target a similar idea: a user should not be
 * able to overwrite an admin file by writing its path in a file form.
 *
 * Comments:
 * (1) The first loop iterates over all the actions of an input sequence.
 * (2) Stores the x-th action of the Input(1) in a variable.
 * (3) Stores the random file path in a variable.
 * (4) Stores the form inputs of the action in a variable.
 * (5) The second loop looks for an action that contains a form that appear to be used to specify
 * paths.
 * (6) Stores the input object of the i-th action in a variable.
 * (7) Verifies that the follow-up user is not an admin (admin may access any file).
 * (8) Verifies that the selected text input in a form contains a file path (or a file name).
 * (9) Verifies defines a follow-up input that is a copy of the source input.
 * (10) Puts a randomly selected path of an admin file in the selected form input of the follow-
 * up sequence.
 * (11) Checks if the y-th action from the source input leads to an error page
 * Or the output generated by sequence containing the path to the admin file is different
 * than the output of the sequence performed by the original user. We do not check only
 * for the output of action x because the error might be observed afterwards (e.g., during
 * execution).
 */

MR CWE_276_277_OTG_AUTHZ_002d {
{
for ( var x=0; Input(1).containFormInputForFilePath() && x<Input(1).actions().size; x++)//(1)
{
var action = Input(1).actions.get(x); // (2)
var randomPath = RandomAdminFilePath(); // (3)
var formInputs = action.formInputs; // (4)
for(var i=0; action.containFormInputForFilePath() && i< formInputs.size(); i++) // (5)
{
var formInput = action.getFormInputs().get(i).getAsJsonObject(); // (6)
IMPLIES(
! isAdmin(action.user) && // (7)
isFormInputForFilePath( formInput ) && // (8)
CREATE (Input(2), Input(1)) &&
updateStringFormInput(Input(2).actions.get(x).getFormInputs().get(i)
.getAsJsonObject(), randomPath) // (10)

```

```
        ,OR(                                                    //(11)
            isError(Output(Input(1))),
            different(Output(Input(1)),Output(Input(2))))
    ); //end-IMPLIES
} //end-for
} //end-for
}
} //end-MR
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/*
 *
 * CWE Definition:
 * CWE_280: The application does not handle or incorrectly handles when it has insufficient
 * privileges to access resources or functionality as specified by their permissions. This may
 * cause it to follow unexpected code paths that may leave the application in an invalid state.
 *
 * CWE_755: The software does not handle or incorrectly handles an exceptional condition.
 *
 * Idea:
 * The application does not handle or incorrectly handles when it has insufficient privileges to
 * access resources or functionality as specified by their permissions. This may cause it to
 * follow unexpected code paths that may leave the application in an invalid state.
 * Assuming certain pages can only be accessed in a certain order, we try to access reserved
 * pages just before the legitimate user may access them.
 *
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1)
 * (2) The second for loop iterates over the sequence of actions after the login.
 * (3) Checks whether the user in User() is not a supervisor of the user performing the current
 * action.
 * (4) Checks that the URL of the current action is not accessible without login.
 * (5) Creates the follow-up input, named Input(2) that takes the login action.
 * (6) Defines the follow-up input, named Input(3) but with different credentials than the
 * Input(2).
 * (7) Creates a copy of Input(1) with a login as b before action y (this way action y is
 * performed as User b).
 * (8) The system should show an error page or the user should see the content that has right to
 * access.
 *
 */

MR CWE_280_755_OTG_AUTHZ_002e{
{
    for(var x = 0; (x < Input(1).actions().size() ); x++){ // (1)
        for (var y = x+1; (isLogin(Input(1).actions().get(x)) && y < Input(1).actions().size());
            y++) { // (2)
            IMPLIES(
                !isSupervisorOf(User(), Input(1).actions().get(y).user) && // (3)
                cannotReachThroughGUI(User(), Input(1).actions().get(y).getUrl()) && // (4)
                CREATE(Input(2), Input(1).actions().get(x)) && // (5)
                CREATE(Input(3), changeCredentials(Input(2), User())) && // (6)
                CREATE(Input(4), addAction(Input(1), y, Input(3).actions().get(0))) // (7)
            ,OR( // (8)
                Output(Input(4), y+1).isError(),
                userCanRetrieveContent(Input(4).actions().get(y+1).getUser(), Output(Input(4),
                    y+1))
            ); //end-IMPLIES
        }
    }
}

```

```
    } //end-for  
  } //end-for  
}  
} //end-MR  
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 * CWE Definition:
 * CWE_284:
 * The software does not restrict or incorrectly restricts access to a resource from an
 * unauthorized actor.
 *
 * CWE_346:
 * The software does not properly verify that the source of data or communication is valid.
 *
 * Idea:
 * Without being authenticated, a user should not be able to access a page
 * that normally can be reached only through the user interface of authenticated users.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Checks that the action is taken after the log in.
 * (3) Verifies that the current action has ever been performed by a non authenticated user.
 * (4) Defines a follow-up input that performs only the given action, without logging in before.
 * (5) Checks if the output generated by the action is different in the two cases.
 *
 */
MR CWE_284_346_OTG_AUTHN_004 {
{
    for ( Action action : Input(1).actions() ) {                                //(1)
        IMPLIES(
            afterLogin( action )&&                                              //(2)
            notVisibleWithoutLoggingIn( action.getUrl() )&&                    //(3)
            CREATE( Input(2) , action )                                          //(4)
            ,
            different(Output(Input(1),action.position),Output(Input(2),action.position)) //(5)
        );//end-IMPLIES
    }//end-for
}
} //end-MR
} //end-package

```

```
import static smrl.mr.language.Operations.*
```

```
package smrl.mr.owasp {
```

```
/**
 * CWE Definition:
 * The software does not properly manage a user within its environment.
 *
 * Idea:
 * A URL that cannot be reached by a user while navigating the user interface should not
 * be available to that same user even when she directly requests the URL to the server.
 *
 * Comments:
 * (1) The for loop iterates over all the actions of an input sequence.
 * (2) Checks whether the user in User() is not a supervisor of the user performing the y-th
 * action.
 * (3) Verifies that the y-th action is performed after a login.
 * (4) Verifies that the follow-up user cannot retrieve the URL of the action through the GUI
 * (based on the data collected by the crawler).
 * (5) Defines a follow-up input that performs the login as the follow-up user
 * (6) The system checks if the y-th action from the source input leads to an error page
 * Or the output generated by the action containing the URL indicated above, lead to two
 * different outputs in the two cases.
 */
```

```
MR CWE_286_OTG_AUTHZ_002c {
{
  for(var y = Input(1).actions().size()-1; ( y > 0 ); y--){
    IMPLIES(
      !isSupervisorOf(User(), Input(1).actions().get(y).user) && // (2)
      afterLogin(Input(1).actions().get(y)) && // (3)
      cannotReachThroughGUI(User(), Input(1).actions().get(y).getUrl()) && // (4)
      CREATE(Input(2), Input(LoginAction(User()), Input(1).actions().get(y)) ) // (5)
    ,
    OR( // (6)
      isError(Output(Input(1), y)),
      different(Output(Input(1), y), Output(Input(2), 1)))
    ); //end-IMPLIES
  } //end-for
}
} //end-MR
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_287:
 * When an actor claims to have a given identity, the software does not prove or insufficiently
 * prove that the claim is correct.
 *
 * CWE_425:
 * The web application does not adequately enforce appropriate authorization on all restricted
 * URLs, scripts, or files.
 *
 * Idea:
 * A login operation should not succeed if performed on the HTTP channel.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1)
 * (2) Stores the parameters of the current action in a variable
 * (3) It checks that the action is a login.
 * (4) It verifies that this login URL was not already tested.
 * (5) Creates the follow-up input by copying the input(1).
 * (6) Sets the HTTP channel for the follow-up input.
 * (7) A login operation should not succeed if performed on the HTTP channel.
 *     checks if the output generated by the login operation is different in the two cases.
 */

MR CWE_287a_425_OTG_AUTHN_001 {
{
    for ( Action action : Input(1).actions() ) {                                //(1)
        var pos = action.getPosition();                                           //(2)
        IMPLIES(
            isLogin(action) &&                                                    //(3)
            notTried(action.url)&&                                                //(4)
            CREATE ( Input(2), Input(1) )    &&                                  //(5)
            Input(2).actions.get(action.position).setChannel("http")            //(6)
            ,
            different ( Output(Input(1),pos),  Output(Input(2),pos) )            //(7)

        );//end-IMPLIES
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp{

/**
 *
 * CWE Definition:
 * CWE_287:
 * When an actor claims to have a given identity, the software does not prove or insufficiently
 * prove that the claim is correct.
 *
 * CWE_288(child of CWE_287):
 * A product requires authentication, but the product has an alternate path or channel that does
 * not require authentication.
 *
 * CWE_300:
 * The product does not adequately verify the identity of actors at both ends of a communication
 * channel or does not adequately ensure the integrity of the channel, in a way that allows the
 * channel to be accessed or influenced by an actor that is not an endpoint.
 *
 * CWE_304:
 * The software implements an authentication technique, but it skips a step that weakens the
 * technique.
 *
 * CWE_319:
 * The software transmits sensitive or security-critical data in cleartext in a communication
 * channel that can be sniffed by unauthorized actors.
 *
 * Idea:
 * All above mentioned CWE weaknesses cover the idea of without being authenticated, a user
 * should not be able to access a page that normally can be reached only through the user
 * interface of authenticated users. This should be true even if she tries on the http channel
 * (i.e., the result of a same operation being performed on a different channel should be
 * different).
 *
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Stores the parameters of the current action in a variable.
 * (3) It checks that the action is taken after the login.
 * (4) Creates the follow-up input.
 * (5) Sets the channel of the follow-up input's action to "http".
 * (6) Checks if the output generated by the action is different in the two cases.
 *
 */

MR CWE_288_287b_300_304_319_OTG_AUTHN_010 {
{
    for ( Action action : Input(1).actions() ){                //(1)
        var pos = action.position;                               //(2)
        IMPLIES(
            afterLogin( action ) &&                               //(3)
            CREATE ( Input(2), Input(1) )&&                       //(4)
            Input(2).actions().get(pos).setChannel("http")        //(5)
        )
    }
}

```

```
        ,
        different( Output(Input(1),pos), Output(Input(2),pos) )    //(6)
    );//end-IMPLIES
} //end-for
}
} //end-MR
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/*
 *
 * CWE Definition:
 * The software performs authentication based on the name of a resource being accessed,
 * or the name of the actor performing the access, but it does not properly check all possible
 * names for that resource or actor.
 *
 * Idea:
 * Protection mechanism that restricts URL access can be bypassed using URL with Hashed
 * separators.
 *
 * comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Stores the parameters of the current action in a variable.
 * (3) Stores the URL of the current action in a variable.
 * (4) Considers users that only have lower or similar access level to Input(1).
 * (5) Filters the dynamic content.
 * (6) Ensure the follow-up user does not have access to the URL.
 * (7) Creates the follow-up Input by copying Input(1) with different credentials, named
 * Input(2).
 * (8) Sets the encoded URL for input(2) to check that the URL action which is not accessible to
 * the Input(2) would be accessible if it is encoded or not.
 * (9) The system should show a content that the user has right to access
 * Or an error page or different outputs for two inputs.
 */

MR CWE_289a {
{
for (Action action: Input(1).actions()){ // (1)
var pos = action.getPosition(); // (2)
var url = action.getUrl(); // (3)
IMPLIES(
!isSupervisorOf(User(), action.user) && // (4)
userCanRetrieveContent(action.getUser(),Output(Input(1) , pos)) && // (5)
cannotReachThroughGUI(User(),action.url) && // (6)
CREATE(Input(2), changeCredentials(Input(1), User())) && // (7)
Input(2).actions.get(action.position).setUrl(EncodeUrl(url)) // (8)
,
OR(userCanRetrieveContent(action.getUser(),Output(Input(2) , pos)) // (9)
,
OR( different( Output( Input(1), pos) , Output(Input(2) , pos) ),
isError( Output(Input(2) , pos)))
)); //end-IMPLIES
} //end-for
}
} //end-MR
} //end-package

```

```
import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/*
 *
 * CWE Definition:
 * CWE_289:
 * The software performs authentication based on the name of a resource being accessed,
 * or the name of the actor performing the access, but it does not properly check all possible
 * names for that resource or actor.
 *
 * CWE_647:
 * The software defines policy namespaces and makes authorization decisions based on the
 * assumption that a URL is canonical. This can allow a non-canonical URL to bypass the
 * authorization.
 *
 * Idea:
 * Protection mechanism that restricts URL access can be bypassed using URL encoding.
 * For example if the "http://URL/users" is not accessible the "http://URL%5Cusers"
 * should not be accessible as well.
 *
 * comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Stores the parameters of the current action in a variable.
 * (3) It creates a new URL address with the hashed format of the separator.
 * (4) Checks if the current action has been performed after a login.
 * (5) Checks if the current user does not have any errors.
 * (6) Defines a follow-up input that is a copy of the source input.
 * (7) Sets the modified URL for the follow-up input.
 * (8) Verifies that the given URL was not tried in a previous execution of the loop (to speed
 * up).
 * (9) The system should not allow the follow-up input to access the content of the URL that is
 * not accessible by the original URL. It should show an error page or a content that the
 * user has already right to access.
 */

MR CWE_289b_647{
{
    var sep = "%5C";
    for ( var par=0; par < 3; par++ ){
        for ( Action action : Input(1).actions() ){ // (1)
            var pos = action.getPosition(); // (2)
            var newUrl = action.rephraseURL(action.urlPath, sep); // (3)
            IMPLIES(
                afterLogin(action) && // (4)
                isError( Output(Input(1) , pos) ) && // (5)
                CREATE( Input(2), Input(1) ) && // (6)
                Input(2).actions().get(pos).setUrl( newUrl ) && // (7)
                notTried( action.getUser(), newUrl ) // (8)
            ),
            OR( // (9)
```

```
        isError( Output(Input(2) , pos)),
        userCanRetrieveContent(action.getUser(),Output(Input(2) , pos)))

    );//end-IMPLIES
} //end-for
sep=sep+"./";
} //end-for
}
} //end-MR
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import smrl.mr.language.CookieSession;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_290:
 * This attack-focused weakness is caused by improperly implemented authentication schemes that
 * are subject to spoofing attacks.
 *
 * CWE_291 (Child of CWE_291):
 * The software uses an IP address for authentication.
 *
 * IP addresses can be easily spoofed. Attackers can forge the source IP address of the packets
 * they send, but response packets will return to the forged IP address. To see the response
 * packets, the attacker has to sniff the traffic between the victim machine and the forged IP
 * address. In order to accomplish the required sniffing, attackers typically attempt to locate
 * themselves on the same subnet as the victim machine. Attackers may be able to circumvent this
 * requirement by using source routing, but source routing is disabled across much of the
 * Internet today.
 *
 * Idea:
 * In summary, IP address verification can be a useful part of an authentication scheme, but it
 * should not be the single factor required for authentication. Then, we will check that for an
 * action with valid IP (user can take the action) if we set a null session the system will
 * authenticate the user or not.
 *
 * Comments:
 * (1) For loop iterates over all Input(1)'s actions
 * (2) Stores the parameters of the current action in a variable.
 * (3) It makes sure that the action is available after the login.
 * (4) Filters the web pages with dynamic content.
 * (5) Makes sure the session of the current action is not null.
 * (6) Creates the follow-up input by copying the Input(1), named Input(2).
 * (7) Sets a null session for the follow-up input.
 * (8) Filter out the refresh action.( makes sure the user does not perform the same action in
 * the next iteration)
 * (9) Verifies that the system would show a content that the user has right to access
 * Or it would show an error page or the user will see a different result than1 the source
 * input.
 */

MR CWE_290_291{
{
    for ( Action action : Input(1).actions() ) { // (1)
        var pos = action.getPosition(); // (2)
        IMPLIES(
            afterLogin(action)&& // (3)

```

```

        userCanRetrieveContent(action.getUser(), Output(Input(1),pos))&&           //(4)
        Input(1).actions.get(pos).session !== null &&                             //(5)
        CREATE ( Input(2), Input(1) ) &&                                           //(6)
        Input(2).actions.get(pos).setSession( new CookieSession() ) &&           //(7)
        !sameUrl( action.currentURL, action.elementURL )                           //(8)
    OR( ,                                                                           //(9)
        isError(Output(Input(2), pos)),
        different ( Output(Input(1),pos), Output(Input(2),pos)))

    );//end-IMPLIES
} //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import sun.security.util.Password

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_298:
 * A certificate expiration is not validated or is incorrectly validated, so trust may be
 * assigned to certificates that have been abandoned due to age.
 *
 * CWE_599:
 * The software uses OpenSSL and trusts or uses a certificate without using the
 * SSL_get_verify_result() function to ensure that the certificate satisfies all necessary
 * security requirements.
 *
 * Idea:
 * we can connect to a webserver after we have a certificate we can simply check if we can
 * connect when we have an outdated certificate. Therefore, after login we will check if we can
 * perform the action or not.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1)
 * (2) Stores the parameters of the current action in a variable
 * (3) It checks if the current action of the Input(1) is the login.
 * (4) Verifies that the current URL has not been seen before, to speed up the process.
 * (5) Filters the webpages with dynamic content.
 * (6) Creates the follow-up input.
 * (7) Sets an out dated certificate for the user.
 * (8) The system should show the error
 * Or the follow-up input should retrieve the content that has right to access
 * Or the result should be different.
 *
 */

MR CWE_298_599 {
{
    for ( Action action : Input(1).actions() ) {                                //(1)
        var pos = action.getPosition();                                           //(2)
        IMPLIES(
            isLogin(action) &&                                                    //(3)
            notTried( action.url, Input(1).actions().get(pos).getElementURL() ) && //(4)
            userCanRetrieveContent(action.user, Output(Input(1), pos))&&         //(5)
            CREATE ( Input(2), Input(1) ) &&                                     //(6)
            Input(2).setUseOutdatedCertificate(true)                             //(7)
        ,
            OR(                                                                    //(8)
                isError(Output(Input(2), pos)) ,
                different ( Output(Input(1),pos), Output(Input(2),pos)))
            );//end-IMPLIES
        }//end-for
    }
}

```

```
//end-MR
//end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import java.util.Map.Entry;
import org.openqa.selenium.Cookie;
import smrl.mr.language.CookieSession;

package smrl.mr.owasp {

/*
 * CWE definition:
 * CWE_302
 * The authentication scheme or implementation uses key data elements that are assumed to be
 * immutable, but can be controlled or modified by the attacker.
 *
 * CWE_471:
 * The software does not properly protect an assumed-immutable element from being modified by an
 * attacker.
 *
 * CWE_472(child of CWE_471):
 * The web application does not sufficiently verify inputs that are assumed to be immutable but
 * are actually externally controllable, such as hidden form fields.
 *
 * CWE_784:
 * The application uses a protection mechanism that relies on the existence or values of a
 * cookie, but it does not properly ensure that the cookie is valid for the associated user.
 *
 * CWE_807:
 * The application uses a protection mechanism that relies on the existence or values of an
 * input, but the input can be modified by an untrusted actor in a way that bypasses the
 * protection mechanism.
 *
 * Idea:
 * Focus on cookies as the key data:
 * Set the cookie path to a random cookie to check whether the system authenticates the user or
 * not.
 *
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Stores the parameters of the current action in a variable.
 * (3) Saves the cookie value of the Input(1) in a variable.
 * (4) Creates a follow-up input with different credentials.
 * (5) Saves the cookie value of the follow-up user in a variable.
 * (6) To speed up the process, verifies that the element URL has not been tried before.
 * (7) Stores the session key values in a variable.
 * (8) For loop iterates over the session values.
 * (9) Assigns the type of the cookie to a variable.
 * (10) Filters all cookie types other than Boolean
 * (11) Makes sure the URL is not accessible without login.
 * (12) To avoid False positives, filters the actions with the same output for Input(1) and
 *      Input(2).
 * (13) Creates the follow-up input, named Input(3).
 * (14) Sets the session2 as the session value of the Input(3).

```

```

* (15) Flip the value of the boolean cookie.
* (16) The system should give a different output or should show an error.
*
*/

```

```

MR CWE_302_471_472_784_807{
{
  for (Action action: Input(1).actions()){                                //(1)
    var pos = action.getPosition();                                        //(2)
    var session = Output( Input(1), pos).session as CookieSession;      //(3)
    CREATE( Input(2), changeCredentials(Input(1), User()))                //(4)
    var session2 = Output( Input(2), pos).session as CookieSession;      //(5)
    var notTried = notTried( action.url, Input(1).actions().get(pos).getElementURL()) //(6)
    var mappings = session.keyValueMappings.entrySet;                    //(7)
    for ( Entry<String,String> cookie : mappings){                        //(8)
      var type = typeOf(cookie.value);                                    //(9)
      IMPLIES(
        notTried &&
        type == Boolean &&                                              //(10)
        cannotReachThroughGUI( User(), action.url) &&                  //(11)
        different( Output( Input(1), pos) , Output(Input(2) , pos)) &&  //(12)
        CREATE ( Input(3), Input(2)) &&                                //(13)
        Input(3).actions.get(pos).setSession( session2 ) &&           //(14)
        session.setCookie(
          new Cookie(cookie.key,String.valueOf(!Boolean.valueOf(cookie.value)))) //(15)
        ,
        OR(                                                              //(16)
          different( Output( Input(1), pos) , Output(Input(3) , pos)),
          isError( Output(Input(3) , pos)))
        );//end-IMPLIES
      }//end-for
    }//end-for
  }
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*

package smrl.mr.owasp {

    /**
     *
     * CWE Definition:
     * The software does not perform any authentication for functionality that requires a provable
     * user identity or consumes a significant amount of resources.
     *
     * Idea:
     * If a certain action is not available to a given user, this user should not be able to
     * perform the action. Assume we have two users, user a and user b. Given (1) a source input as
     * a sequence of actions performed by a user 'a' which contains an action y that is dedicated
     * to user a (i.e., it is not visible to user b) and (2) a follow-up input that is a copy of
     * that sequence which, however, includes, before action y, an action that matches action y
     * (e.g., same URL requested) but is performed by user 'b'. The result of action y should not
     * be different when performed in the source input (i.e., without any action of b) or in the
     * follow-up input (i.e., when performed also by user b).
     * In other words, the action of user b should be ignored by the system and not interfere with
     * the action of user a.
     *
     * Comment:
     * (1) The first loop iterates over the actions of the source input to identify a login
     * operation (action x) for user a,
     * (2) The second loop iterates over the remaining y-th actions.
     * (3) Checks whether the user in User() is not a supervisor of the user performing the y-th
     * action.
     * (4) Checks that action y cannot be accessed by user b (User()).
     * (5) Defines Input(2) which just performs a login.
     * (6) Defines Input(3) which just performs a login as user b.
     * (7) Creates a copy of Input(1) with a login as b before action y (this way action y is
     * performed as User b).
     * (8) Adds after action y+1 (the original action y now shifted) new copy of action y (now
     * performed by user b).
     * (9) Adds after the new copy of action y a new login as user a.
     * (10) The system checks that the output of the action y in the two sequences remains the same
     * when performed by user a (in the follow-up sequence the action of user a is shifted by
     * three because three actions are introduced, the login of user b, the current action and
     * a new login for user a).
     */

    MR CWE_306_OTG_AUTHZ_002b {
        {
            for(var x = 0; x < Input(1).actions().size() ; x++){
                for (var y = x+1; isLogin(Input(1).actions().get(x)) && (y < Input(1).actions().size());
                    y++) {
                    IMPLIES(
                        !isSupervisorOf(User(), Input(1).actions().get(y).user) &&
                        cannotReachThroughGUI(User(), Input(1).actions().get(y).getUrl()) &&
                        CREATE(Input(2), Input(1).actions().get(x)) &&
                        CREATE(Input(3), changeCredentials(Input(2), User())) &&
                        CREATE(Input(4), addAction(Input(1), y, Input(3).actions().get(0))) &&
                        CREATE(Input(5), addAction(Input(4), y+1, Input(1).actions().get(y))) &&
                    // (1)
                    // (2)
                    // (3)
                    // (4)
                    // (5)
                    // (6)
                    // (7)
                    // (8)
                }
            }
        }
    }
}

```

```

        CREATE(Input(6), addAction(Input(5), y+2, Input(1).actions().get(x)))          //(9)
    ,
    EQUAL(                                                                              //(10)
        Output(Input(1), y) ,
        Output(Input(6), y+3 ))
    ); //end-IMPLIES
} //end-for
} //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import java.util.function.Predicate;
import java.util.stream.Collectors;

package smrl.mr.owasp{

/**
 * CWE Definitions:
 *
 * CWE_312:
 * The application stores sensitive information in cleartext within a resource that might be
 * accessible to another control sphere.
 *
 * CWE-313:
 * The application stores sensitive information in cleartext in a file, or on disk.
 *
 * CWE_532 (Child of CWE_538):
 * Information written to log files can be of a sensitive nature and give valuable guidance to an
 * attacker or expose sensitive user information.
 *
 * CWE_538:
 * The product places sensitive information into files or directories that are accessible to
 * actors who are allowed to have access to the files, but not to the sensitive information.
 *
 * CWE-359:
 * The product does not properly prevent a person's private, personal information from being
 * accessed by actors who either (1) are not explicitly authorized to access the information or
 * (2) do not have the implicit consent of the person about whom the information is collected.
 *
 * Idea:
 * When we identify a page that can be accessed only by a user, then it means that the contained
 * information is reserved, thus the log generated when processing it should not contain reserved
 * information.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Creates a cache of the current status of the log file.
 * (3) Verified that current action follows a login.
 * (4) Creates a follow-up input, named Input(2).
 * (5) Verifies that the current action is reserved to the current user (i.e., a follow-up user
 * cannot perform it)
 * (6) Verifies that the current URL leads to some output
 * (7) Verifies that the that new lines generated into the log file when processing the input
 * (7a) do not include keywords that appear only in pages of the user and in particular in
 * the page generated for the current action (7b).
 */

```

```

MR CWE_312_313_532_538_359 {
{
  for (Action action : Input(1).actions()){ // (1)
    var initialLog = Log(); // (2)
    IMPLIES(
      afterLogin(action) && // (3)
      CREATE(Input(2), Input(1)) && // (4)
      cannotReachThroughGUI(User(), action.url) && // (5)
      Output(Input(2), action.position).values.size > 0 // (6)
    ,
      NOT ( containsAny(Log().newLines(initialLog), // (7a)
        Output(Input(2), action.position).values.filter[ e |
          reservedKeywords(action.user).contains(e) ].toList() ) ) // (7b)
    ); //end-IMPLIES
  } //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/*
 *
 * CWE Definition:
 * The application stores sensitive information in cleartext in the registry.
 *
 * Idea:
 * If the system stores critical information in cleartext and its parameters
 * are vulnerable to long strings then we can give random strings as input and
 * let the system to crash and reveal the content. In this regard we will give
 * random strings with three different lengths including 512, 1024, and 2048 to
 * check whether the system show the critical content to the user or not.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Stores the parameters of the current action in a variable.
 * (3) Defines random string with three different lengths.
 * (4) Second for loop iterates over all parameters of the current action.
 * (5) Makes sure the current action does not have any alerts.
 * (6) Filters the dynamic content.
 * (7) Verifies that the current URL has not been seen before
 * (8) Creates the follow-up input, named Input(2).
 * (9) Sets a random string with 512 characters to the parameter of the action
 * (10) Creates the follow-up input, named Input(3).
 * (11) Sets a random string with 1024 characters to the parameter of the action
 * (12) Creates the follow-up input, named Input(4).
 * (13) Sets a random string with 2048 characters to the parameter of the action
 * (14) There is OR conditions among three different follow-up inputs
 *      the reason is the system may crash or reveal the content with different string lengths
 *      we check that the result of the follow-up input is error
 *      Or the follow-up input can retrieve the content that has the right to access
 *      Or either the attack was not performed
 *
 */

MR CWE_314{
{
    for (Action action : Input(1).actions()){                                //(1)
        var pos = action.getPosition();                                        //(2)
        var str512 = action.randomString(512);                             //(3)
        var str1024 = action.randomString(1024);
        var str2048 = action.randomString(2048);
        for (var x = 0; x < action.parameters.size; x++){                    //(4)
            IMPLIES(
                !Output(Input(1), pos).hasAlert    &&                        //(5)
                userCanRetrieveContent(action.getUser(), Output(Input(1),pos))    &&    //(6)
                notTried( Input(1).actions().get(pos).getUrl() ) &&            //(7)
                CREATE(Input(2), Input(1)) &&                                //(8)
                Input(2).actions().get(pos).setParameterValue( x, str512 )&&    //(9)
            )
        }
    }
}

```

```

CREATE(Input(3), Input(1)) &&                                     //(10)
Input(3).actions().get(pos).setParameterValue( x, str1024 )&&    //(11)
CREATE(Input(4), Input(1)) &&                                     //(12)
Input(4).actions().get(pos).setParameterValue( x, str2048 )      //(13)
',
OR( OR(                                                           //(14)
    OR(
        OR(
            isError(Output(Input(2), pos)),
            userCanRetrieveContent(action.getUser(), Output(Input(2),pos))
        ),
        OR(
            Output(Input(2), pos).emptyFile,
            !Output(Input(2), pos).hasAlert)
        ),
    OR(
        OR(
            isError(Output(Input(3), pos)),
            userCanRetrieveContent(action.getUser(), Output(Input(3),pos))
        ),
        OR(
            Output(Input(3), pos).emptyFile,
            !Output(Input(3), pos).hasAlert)
        ),
    ),
OR(
    OR(
        isError(Output(Input(4), pos)),
        userCanRetrieveContent(action.getUser(), Output(Input(4),pos))
    ),
    OR(
        Output(Input(4), pos).emptyFile,
        !Output(Input(4), pos).hasAlert )
    ),
); //end-IMPLIES
}} //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import sun.security.util.Password

package smrl.mr.owasp {

/**
 * CWE Definition:
 * The application stores sensitive information in cleartext in a cookie.
 *
 * Attackers can use widely-available tools to view the cookie and read the sensitive
 * information. Even if the information is encoded in a way that is not human-readable, certain
 * techniques could determine which encoding is being used, then decode the information.
 *
 * Idea:
 * We assume that even if the password is encrypted it can be identified, i.e. by exploring
 * various encryption algorithms Then the attacker should not be allowed to access the password
 * field of the cookie.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Stores the parameters of the current action in a variable.
 * (3) Stores the cookie of the current action in a variable.
 * (4) Creates the follow-up input.
 * (5) checks that the session value should not contain the user's password.
 *
 */

MR CWE_315 {
{
    for ( Action action : Input(1).actions() ) {                                //(1)
        var pos = action.getPosition();                                           //(2)
        IMPLIES(
            notTried( Input(1).actions().get(pos).getUrl() ) &&                 //(3)
            CREATE ( Input(2), Input(1) )                                         //(4)
            ,
            FALSE ( Output(Input(2),pos).session.keyValueMappings.values
                .contains(action.user.password) )                                //(5)
        );//end-IMPLIES
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import smrl.mr.crawljax.WebProcessor

package smrl.mr.owasp{

/**
 * CWE Definition:
 * The web application does not, or can not, sufficiently verify whether a well-formed,
 * valid, consistent request was intentionally provided by the user who submitted the request.
 *
 * Idea:
 * A user shall not be able to successfully submit a form without retrieving it first.
 * Otherwise, someone else can create a crafted page with a submit form button.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1)
 * (3) Stores the parameters of the current action in a variable
 * (4) Iterates over the actions that contain a form input.
 * (5) Checks if the action submits the form; i.e. the action contains clicking on submit button.
 * (6) Verifies that the URL has not been tried before.
 * (7) Makes sure the action is performed after the login.
 * (8) Creates Input(2) as the follow-up input from the user that takes the login action and
 * (9) It sets the URL of the file (to retrieve the file before uploading it), and the action.
 * (10) The system should show a different output.
 */

MR CWE_352 {
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        var pos = action.getPosition(); // (3)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (4)
            IMPLIES(
                action.isClickOnButton && // (5)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL() ) && // (6)
                afterLogin( action ) && // (7)
                CREATE( Input(2), Input( LoginAction(action.user) , // (8)
                RequestUrlAction("file://" + Output(Input(1),pos-1).file.absolutePath),action ) ) // (9)
                ,
                different ( Output(Input(1),pos), Output(Input(2),3)) // (10)
            );//end-IMPLIES
        }//end-for
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
package smrl.mr.owasp {

/*
 * CWE Definition:
 * The software protects a primary channel, but it does not use the same level of protection for
 * an alternate channel.
 *
 * Idea:
 * An action with strict transport security header should not be available on the http channel.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Stores the parameters of the current action in a variable.
 * (3) creates the follow-up input, named Input(2).
 * (4) Checks if the output of the source input has strict transport security header.
 * (5) Sets the channel of the action to http
 * (6) Checks that if the modified action has not been redirected to https then the output
 *     generated by the action should be different than in the case of the source input.
 *
 */

MR CWE_420_OTG_CONFIG_007 {
{
    for ( Action action : Input(1).actions() ) {                                //(1)
        var pos = action.getPosition();                                           //(2)
        IMPLIES(
            CREATE ( Input(2) , Input(1) &&                                     //(3)
                Output(Input(1),pos).hasStrictTransportSecurityHeader() &&      //(4)
                Input(2).actions().get(pos).setMethod("http") )                //(5)
            ,
            OR(                                                                    //(6)
                AND (
                    equal ( Output(Input(2),pos).getChannel(), "https" ),
                    equal ( Output(Input(1),pos) , Output(Input(2),pos)))
                ,
                isError( Output(Input(2),pos)))
            );//end-IMPLIES
        }//end-for
    }
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import sun.security.util.Password

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_434:
 * The software allows the attacker to upload or transfer files of dangerous types that can be
 * automatically processed within the product's environment.
 *
 * CWE_669:
 * The product does not properly transfer a resource/behavior to another sphere, or improperly
 * imports a resource/behavior from another sphere, in a manner that provides unintended control
 * over that resource.
 *
 * Idea:
 * For all input entries that accept to upload a file, we will try to upload files with invalid
 * types e.g. file.php or file.exe.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1)
 * (3) Stores the parameters of the current action in a variable
 * (4) Iterates over the actions that contain a form input.
 * (5) Checks if the action submits the form; i.e. the action contains clicking on submit button.
 * (6) Verifies that the URL has not been tried before.
 * (7) Checks if the action initially does not have any alerts.
 * (8) Creates Input(2) as the follow-up input.
 * (9) Set the path to a file with invalid type as the form input value.
 * (10) The system should show a different result
 * Or the attack should not be performed.
 *
 */

```

```

MR CWE_434 {
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        var pos = action.getPosition(); // (3)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (4)
            IMPLIES(
                action.isClickOnButton && // (5)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL() ) && // (6)
                !Output(Input(1),pos).hasAlert && // (7)
                CREATE(Input(2), Input(1) ) && // (8)
                Input(2).actions.get(pos).setFormInput(x, FileWithInvalidType()) // (9)
            ,
            OR(

```

```
different( Output(Input(1),pos+1),Output(Input(2),pos+1))           //(10)
,
OR(
    Output(Input(2), pos+2).emptyFile,
    !Output(Input(2), pos+2).hasAlert))
);//end-IMPLIES
} //end-for
} //end-for
}
} //end-MR
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/*
 *
 * CWE Definition:
 * The product does not sufficiently enforce boundaries between the states of different sessions,
 * causing data to be provided to, or used by, the wrong session.
 *
 * Idea:
 * Assuming certain pages can only be accessed in a certain order, we try to access reserved
 * pages just before the legitimate user may access them.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1)
 * (2) The second for loop iterates over the sequence of actions after the login.
 * (3) Checks whether the user in User() is not a supervisor of the user performing the current
 *      action.
 * (4) Checks that the requested web page does not contain dynamic content.
 * (5) Creates the follow-up input, named Input(2) that takes the login action.
 * (6) Defines the follow-up input, named Input(3) but with different credentials than the
 *      Input(2).
 * (7) Creates a copy of Input(1) with a login as b before action y (this way action y is
 *      performed as User b).
 * (8) The system should show an error page, or different content, or a content that the user has
 *      right to access.
 *
 */

MR CWE_488{
{
    for(var x = 0; (x < Input(1).actions().size() ); x++){                                //(1)
        for (var y = x+1; (isLogin(Input(1).actions().get(x)) && y < Input(1).actions().size()); y++)
        {                                                                                //(2)
            IMPLIES(
                !isSupervisorOf(User(), Input(1).actions().get(y).user) &&                //(3)
                userCanRetrieveContent(Input(1), Output(Input(1),y))&&                //(4)
                CREATE(Input(2), Input(1).actions().get(x)) &&                //(5)
                CREATE(Input(3), changeCredentials(Input(2), User())) &&                //(6)
                CREATE(Input(4), addAction(Input(1), y, Input(3).actions().get(0)))        //(7)
            ),
            OR(                                                                           //(8)
                userCanRetrieveContent(Input(4).actions().get(y+1).getUser(), Output(Input(4), y+1))
            ),
            OR( Output(Input(4), y+1).isError(),
                different(Output(Input(1), y+1),Output(Input(4), y+1)))
            ); //end-IMPLIES
        } //end-for
    } //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import sun.security.util.Password
import java.rmi.server.Operation
import smrl.mr.crawljax.WebOutputCleaned

package smrl.mr.owasp {

/**
 * CWE Definition:
 * The product does not require that users should have strong passwords, which makes it easier
 * for attackers to compromise user accounts.
 *
 * Idea:
 * The idea is that a good system should not allow the user to create an account with a weak
 * password. We create a list of weak password i.e. only numbers, only characters, and etc.
 * for each password parameter we will try a weak passwords from our list, and a good system
 * should not allow the user to sign up with that weak password.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1)
 * (2) Store the parameters of the current action in a variable
 * (3) Creates some random values to be used as username for source and follow-up inputs.
 * (4) filters the webpages that contain a form input.
 * (5) Checks that the user will submit the form.
 * (6) It checks if the current action of the Input(1) is signing up.
 * (7) Makes sure that the URL was not seen before, to speed up the process.
 * (8) Creates Input(2) as the follow-up input.
 *
 * To make sure we are targeting only the password field, we are giving some default values to
 * the other input entries.
 * (9) Set the weak value for the password value of the Input(2)
 * (10),(15) Enters the random value for the name.
 * (11),(16) Enters the random value for the username or ID.
 * (12),(17) Enters the random value for the email address.
 * (13) Creates Input(3) as the second follow-up input.
 * (14) Considers a strong password for input(3).
 * (18) The Input(2) should not be able to sign up with a weak password, she should see a
 *      different content or an error page.
 */

MR CWE_521_b {
{
  for ( Action action : Input(1).actions() ) { // (1)
    var pos = action.getPosition(); // (2)
    var randomString = "rand" + (Math.random() * 1000) as int; // (3)
    var randomString_A = randomString + "A";
    var randomString_B = randomString + "B";
    IMPLIES(
      action.containAnyFormInput() && // (4)
      action.isClickOnButton && // (5)
      isSignup(action) && // (6)
      userCanRetrieveContent(action.getUser(), Output(Input(1), pos)) &&
    )
  }
}

```

```

notTried( action.url, Input(1).actions().get(pos).getElementURL() ) && // (7)
CREATE ( Input(2), Input(1) )&& // (8)
Input(2).actions.get(action.position).setFormInputsOfType(WeakPassword(), "password")&& // (9)
Input(2).actions.get(action.position).setFormInputsOfType(randomString, "text") && // (10)
Input(2).actions.get(action.position).setFormInputsParameterValue(
    "text",getSysConfig().getSignupUserParam(),randomString_B) && // (11)
Input(2).actions.get(action.position).setFormInputsParameterValue(
    "text","email","ahg@hhh.com") && // (12)
CREATE ( Input(3), Input(2) ) && // (13)
Input(3).actions.get(action.position).setFormInputsOfType("@_aAn@12", "password") && // (14)
Input(3).actions.get(action.position).setFormInputsOfType(randomString, "text") && // (15)
Input(3).actions.get(action.position).setFormInputsParameterValue("text",getSysConfig()
    .getSignupUserParam(),randomString_A) && // (16)
Input(3).actions.get(action.position).setFormInputsParameterValue("text","email",
    "ahg@hhh.com") // (17)

OR( , // (18)
    different(Output(Input(2), pos),Output(Input(3), pos)),
    isError(Output(Input(2), pos)))

); //end-IMPLIES
} //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import sun.security.util.Password

package smrl.mr.owasp {

/**
 * CWE Definition:
 * The product does not require that users should have strong passwords, which makes it easier
 * for attackers to compromise user accounts.
 *
 * Idea:
 * We create a list of weak password i.e. only numbers, only characters, less than five
 * characters, and etc. for each password parameter we will try the weak passwords from our list,
 * and a good system should not allow the user to sign up with that weak password.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1)
 * (2) Store the parameters of the current action in a variable
 * (3) It checks if the current action of the Input(1) is signing up.
 * (4) Creates Input(2) as the follow-up input.
 * (5) Set the weak value for the password value of the Input(2)
 * (6) The Input(2) should not be able to sign up with a weak password, she should see a
 *     different content or an error page.
 */

MR CWE_521 {
{
    for ( Action action : Input(1).actions()){                                //(1)
        var pos = action.getPosition();                                         //(2)
        IMPLIES(
            isSignup(action) &&                                                //(3)
            CREATE ( Input(2), Input(1)) &&                                    //(4)
            Input(2).actions.get(action.position).setValueForParametersOfType( "password",
                WeakPassword())                                                //(5)

            ,
            OR(                                                                    //(6)
                isError(Output(Input(2), pos)),
                different ( Output(Input(1),pos),  Output(Input(2),pos)))
            );//end-IMPLIES
        }//end-for
    }
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * A web application accepts a user-controlled input that specifies a link to an external site,
 * and uses that link in a Redirect. This simplifies phishing attacks.
 *
 * Idea:
 * If a redirecting URL cannot be reached by a user while navigating the user interface,
 * the same URL, if directly requested to the server, should not
 * enable the same user to access a page where the click on one of its elements
 * (e.g., a warning message) enables the user to access the content of the URL.
 *
 * Comments:
 * (1) The for loop iterates over all actions of the Input(1).
 * (2) Stores the parameters of the current action in a variable.
 * (3) Checks whether the user in User() is not a supervisor of the user performing the current
 *     action.
 * (4) Verifies that the user cannot retrieve the URL of the action through the GUI (based on the
 *     data collected by the crawler).
 * (5) Defines a follow-up input that matches the source input except that the credentials of
 *     User() are used in this case.
 * (6) It is made of three clauses.
 *     The 1st clause verifies that the original URL does not perform any redirect.
 *     The 2nd clause verifies that the original URL does not perform any redirect.
 *     The 3rd clause verifies that the follow up input does not lead to the same redirect from
 *     the original input.
 */

MR CWE_601_OTG_AUTHZ_002a {
{
    for ( Action action : Input(1).actions() ){                                //(1)
        var pos = action.getPosition();                                         //(2)
        IMPLIES(
            !isSupervisorOf(User(), action.user) &&                           //(3)
            cannotReachThroughGUI(User(), action.url) &&                       //(4)
            CREATE( Input(2), changeCredentials( Input(1), User() ) )          //(5)
        ,
            (Output(Input(1), pos).redirectURL()===null ||                      //(6)
             Output(Input(2), pos).redirectURL()===null ) ||
            NOT(EQUAL(
                Output(Input(2), pos).redirectURL(),
                Output(Input(1), pos).redirectURL()))
            ); //end-IMPLIES
        } //end-for
    }
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;

import smrl.mr.language.Action;

package smrl.mr.owasp{

/**
 * CWE Definition:
 *
 * CWE_610:
 * The product uses an externally controlled name or reference that resolves to a resource that
 * is outside of the intended control sphere.
 *
 * CWE_384 (Child of CWE_610):
 * Authenticating a user, or otherwise establishing a new user session, without invalidating any
 * existing session identifier gives an attacker the opportunity to steal authenticated sessions.
 *
 *
 * A web application authenticates a user without first invalidating the existing session,
 * thereby continuing to use the session already associated with the user. An attacker is able to
 * force a known session identifier on a user so that, once the user authenticates, the attacker
 * has access to the authenticated session. The application or container uses predictable session
 * identifiers. In the generic exploit of session fixation vulnerabilities, an attacker creates a
 * new session on a web application and records the associated session identifier. The attacker
 * then causes the victim to associate, and possibly authenticate, against the server using that
 * session identifier, giving the attacker access to the user's account through the active
 * session.
 *
 * Idea:
 * When a user login to the system, the system will create a new session for that person.
 * Even if the user is active and asks for login e.g. from another system or another browser, the
 * system should create the new session for the user.
 * In this MR we are trying to implement this idea.
 * We are looking for two logins in the system (like the examples that user logins from two
 * different systems) without any logout i.e. if the user does the logout, the system
 * automatically will gives a new session; and we are testing the new session after doing a
 * logout in another MR. We will check that after the second login the system assigns a different
 * session to the user or not, also the user should not be able to continue her actions
 * after the second login.
 *
 * Considering that we have a user that is active and the attacker can steal the session.
 * The attacker will use the session to log in, and the system should not validate the session.
 * After the second login the session hijacking system should publish a new session for the user.
 * We shall simply perform a login within another logged in session. No logout is needed.
 *
 *
 * Comments:
 * (1) The first loop iterates over all the actions of the Input(1).
 * (2) The second loop iterates over all the actions after the x to find an action performed
 * after login.
 * (3) Avoids resetting the browser.
 * (4) Filters the actions that initially contain error message.
 * (5) Sets the sub-sequence of actions to Input(2), from beginning to the first action after the

```

```

* login.
* (6) Sets the sub-sequence of actions to Input(3), from the first action after the login till
* the end.
* (7) Makes sure that Input(3) starts with a login.
* (8) Set the same session of Input(2) for the Input(3).
* (9) The system should assign two different session IDs after the log in for Input(2) and
* Input(3)
* And the system should invalidate the session of Input(2) therefore the output of Input(3)
* at position 1 i.e., (the action performed after login) for which we used the session id of
* Input(2)) should be either an error message or different than the output of Input(1) for
* the same input.
*
*/

```

```

MR CWE_610_384 {
{
  for ( var x = 0; x < Input(1).actions().size(); x++ ){                                //(1)
    for ( var y = x+1; isLogin(Input(1).actions().get(x)) && (y < Input(1).actions().size());
        y++){                                                                    //(2)
      setResetBrowserBetweenInputs( false );                                     //(3)
      IMPLIES (
        !isError(Input(1).actions().get(x+1))&&                                //(4)
        CREATE(Input(2), Input(Input(1).actions().subList(0, y))) &&          //(5)
        CREATE(Input(3), Input(Input(1).actions().subList(y, Input(1)
          .actions().size()))&&                                                //(6)
        Input(3).addAction(0,Input(1).actions().get(x)) &&                    //(7)
        Input(3).actions().get(1).setSession( Output(Input(2),x).getSession()) //(8)
        ,
      OR(                                                                        //(9)
        different ( Output(Input(2),x), Output(Input(3),0)),
        AND(
          different(Output(Input(2),x).getSession(), Output(Input(3),0).getSession()),
          isError(Output(Input(3),1))))
      ); //end-IMPLIES
      setResetBrowserBetweenInputs( true );
    } //end-for
  } //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp{

/**
 * CWE Definition:
 * The software processes an XML document that can contain XML entities with URIs that resolve to
 * documents outside of the intended sphere of control, causing the product to embed incorrect
 * documents into its output.
 *
 * XML documents optionally contain a Document Type Definition (DTD), which, among other
 * features, enables the definition of XML entities. It is possible to define an entity by
 * providing a substitution string in the form of a URI. The XML parser can access the contents
 * of this URI and embed these contents back into the XML document for further processing.
 *
 * Idea:
 * For all pages that the user can upload a file we try to upload the malformed XML file.
 * the system should not verify the request in the XML file. It means the system should
 * show the content to the user that has the right to access to it or an error page.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Stores the parameters of the current action in a variable.
 * (4) Second for loop iterates over all parameters of the current action and verifies that the
 * action has input form.
 * (5) Checks that user submits the form.
 * (6) Verifies that the current URL has not been seen before, to speed up the process.
 * (7) Checks that the action originally does not contain any alert.
 * (8) Creates the follow-up input
 * (9) Uploads the malformed XML file to the parameter of the action.
 * (10) Checks that the result of the follow-up input is error
 * Or the follow-up input can has the right to access
 * Or the user will retrieve an empty file or she will see an alert message.
 */

MR CWE_611 {
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        var pos = action.getPosition(); // (3)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (4)
            IMPLIES(
                action.isClickOnButton && // (5)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL() ) && // (6)
                !Output(Input(1),pos).hasAlert && // (7)
                CREATE(Input(2), Input(1) ) && // (8)
                Input(2).actions.get(pos).setFormInput(x, XMLInjectedFile()) // (9)
            ,
            OR( // (10)

```

```

    OR(
      different( Output(Input(1),pos+1),Output(Input(2),pos+1)),
      isError(Output(Input(2),pos+1))  )
    ,
    OR(
      Output(Input(2), pos+1).emptyFile,
      !Output(Input(2), pos+1).hasAlert))
  );//end-IMPLIES
} //end-for
} //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;

package smrl.mr.owasp{

/**
 *
 * CWE Definition:
 * According to WASC, "Insufficient Session Expiration is when a web site permits an attacker to
 * reuse old session credentials or session IDs for authorization."
 * The lack of proper session expiration may improve the likely success of certain attacks.
 * For example, an attacker may intercept a session ID, possibly via a network sniffer or
 * Cross-site Scripting attack. Although short session expiration times do not help if a stolen
 * token is immediately used, they will protect against ongoing replaying of the session ID. In
 * another scenario, a user might access a web site from a shared computer (such as at a library,
 * Internet cafe).
 * Insufficient Session Expiration could allow an attacker to use the browser's back button to
 * access web pages previously accessed by the victim.
 *
 * Idea:
 * A logout action should always lead to a new session.
 *
 * Comments:
 * (1) The first loop iterates over all the actions to find a logout action.
 * (2) The second loop iterates over all the actions to find an action performed after login.
 * (3) Checks if the current action x is a logout operation.
 * (4) Checks that the action y is performed after a login.
 * (5) Checks that the action y is not a login.
 * (6) Defines a follow-up input with the logout action being duplicated in position y.
 * (7) Verifies if the session ID before and after executing the logout is different.
 */
MR CWE_613_OTG_SESS_006 {
{
    for ( var x=0; x < Input(1).actions().size() ; x++ ){                //(1)
        for ( var y=0; y < x ; y++ ){                                    //(2)
            IMPLIES (
                isLogout( Input(1).actions().get(x)) &&                //(3)
                afterLogin( Input(1).actions().get(y)) &&                //(4)
                !isLogin( Input(1).actions().get(y)) &&                //(5)
                CREATE ( Input(2) , copyActionTo( Input(1), x, y ))      //(6)
                ,
                different ( Session(Input(2),y-1), Session(Input(2),y))  //(7)
            ); //end-IMPLIES
        } //end-for
    } //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import smrl.mr.language.actions.StandardAction;
import jdk.internal.util.xml.impl.Input

package smrl.mr.owasp{

/**
 *
 * CWE Definition:
 *
 * CWE_643:
 * The software uses external input to dynamically construct an XPath expression used to
 * retrieve data from an XML database, but it does not neutralize or incorrectly neutralizes that
 * input. This allows an attacker to control the structure of the query.
 *
 *
 * CWE_652:
 * The software uses external input to dynamically construct an XQuery expression used to
 * retrieve data from an XML database, but it does not neutralize or incorrectly neutralizes that
 * input. This allows an attacker to control the structure of the query.
 *
 * OWASP hint:
 * with XPath injection, the first step is to insert a single quote (') in the field to
 * be tested, introducing a syntax error in the query, and to check whether the application
 * returns an error message.
 *
 *
 * Idea:
 * In the login page instead of providing a correct value to the username field, we provide a
 * malicious value ( e.g., "'" or 1=1 or ''=''") and a random value as a wrong password. If the
 * system is vulnerable it may give access to the attacker. Therefore, if the system is not
 * vulnerable it should show an error page or a content that the user has right to access.
 * Since every login interface provided by a different URL may be handled by a different
 * implementation, in this MR, we perform the attack for every login action URL observed.
 * The idea behind the CWE_643 that is about XPath is similar to the XQuery injection in CWE_652.
 * XQuery supports XPath and extended relational models. According to the real cases, we can
 * trigger the weakness with the same inputs, then we merged these two CWE vulnerabilities
 * together.
 *
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1)
 * (3) Stores the parameters of the current action in a variable
 * (4) Iterates over the actions that contain a form input.
 * (5) Checks if the action submits the form; i.e. the action contains clicking on submit button.
 * (6) Filters the dynamic content.
 * (7) Verifies that the URL has not been tried before.
 * (8) Creates Input(2) as the follow-up input.
 * (9) Set the XQuery string as the form input value.
 * (10) The system should show an error page or a content that the user has right to access.
 */
}

```

```

MR CWE_643_652_B {
{
  keepDialogsOpen = true; // (1)
  for (Action action : Input(1).actions()){ // (2)
    var pos = action.getPosition(); // (3)
    for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (4)
      IMPLIES(
        action.isClickOnButton && // (5)
        userCanRetrieveContent(action.getUser(), Output(Input(1),pos))&& // (6)
        notTried( action.url, Input(1).actions().get(pos).getElementURL() ) && // (7)
        CREATE(Input(2), Input(1) ) && // (8)
        Input(2).actions.get(pos).setFormInput(x, XQueryInjection()) // (9)
      ,
      OR( // (10)
        isError(Output(Input(2), pos)),
        userCanRetrieveContent(action.getUser(), Output(Input(2),pos))
      ); //end-IMPLIES
    } //end-for
  } //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 *
 * CWE_643:
 * The software uses external input to dynamically construct an XPath expression used to
 * retrieve data from an XML database, but it does not neutralize or incorrectly neutralizes that
 * input. This allows an attacker to control the structure of the query.
 *
 *
 * CWE_652:
 * The software uses external input to dynamically construct an XQuery expression used to
 * retrieve data from an XML database, but it does not neutralize or incorrectly neutralizes that
 * input. This allows an attacker to control the structure of the query.
 *
 * OWASP hint:
 * with XPath injection, the first step is to insert a single quote (') in the field to
 * be tested, introducing a syntax error in the query, and to check whether the application
 * returns an error message.
 *
 *
 * Idea:
 * In the login page instead of providing a correct value to the username field, we provide a
 * malicious value ( e.g., "'" or 1=1 or ''=''") and a random value as a wrong password. If the
 * system is vulnerable it may give access to the attacker. Therefore, if the system is not
 * vulnerable it should show an error page or a content that the user has right to access.
 * Since every login interface provided by a different URL may be handled by a different
 * implementation, in this MR, we perform the attack for every login action URL observed.
 * The idea behind the CWE_643 that is about XPath is similar to the XQuery injection in CWE_652.
 * XQuery supports XPath and extended relational models. According to the real cases, we can
 * trigger the weakness with the same inputs, then we merged these two CWE vulnerabilities
 * together.
 *
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1)
 * (3) Stores the parameters of the current action in a variable
 * (4) Iterates over the actions that contain a login form input.
 * (5) Checks if the action submits the form; i.e. the action contains clicking on submit button.
 * (6) Filters the dynamic content.
 * (7) Verifies that the URL has not been tried before.
 * (8) Verifies the output of the action does not contain any errors.
 * (9) Creates Input(2) as the follow-up input.
 * (10) Set the XQuery string as the form input value.
 * (11) The system should show an error page or a content that the user has right to access.
 *

```

```

*/

MR CWE_643_652_C {
{
    keepDialogsOpen = true; // (1)
    for ( Action action : Input(1).actions()){ // (2)
        var pos = action.getPosition(); // (3)
        for (var x = 0; action.containLoginFormInput() && x < action.formInputs.size; x++){ // (4)
            IMPLIES(
                action.isClickOnButton && // (5)
                userCanRetrieveContent(action.getUser(), Output(Input(1),pos)) && // (6)
                notTried(action.url, Input(1).actions().get(pos).getElementURL()) && // (7)
                !isError(Output(Input(1),pos)) && // (8)
                CREATE ( Input(2), Input(1) ) && // (9)
                Input(2).actions.get(action.position).setUsernameFormInput(x, XQueryInjection()) // (10)
            ,
            OR( // (11)
                isError(Output(Input(2), pos)),
                userCanRetrieveContent(action.getUser(), Output(Input(2),pos)))
            ); //end-IMPLIES
        } //end-for
    } //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import sun.security.util.Password

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 *
 * CWE_643:
 * The software uses external input to dynamically construct an XPath expression used to
 * retrieve data from an XML database, but it does not neutralize or incorrectly neutralizes that
 * input. This allows an attacker to control the structure of the query.
 *
 *
 * CWE_652:
 * The software uses external input to dynamically construct an XQuery expression used to
 * retrieve data from an XML database, but it does not neutralize or incorrectly neutralizes that
 * input. This allows an attacker to control the structure of the query.
 *
 * OWASP hint:
 * with XPath injection, the first step is to insert a single quote (') in the field to
 * be tested, introducing a syntax error in the query, and to check whether the application
 * returns an error message.
 *
 *
 * Idea:
 * In the login page instead of providing a correct value to the username field, we provide a
 * malicious value ( e.g., "'" or 1=1 or ''=''") and a random value as a wrong password. If the
 * system is vulnerable it may give access to the attacker. Therefore, if the system is not
 * vulnerable it should show an error page or a content that the user has right to access.
 * Since every login interface provided by a different URL may be handled by a different
 * implementation, in this MR, we perform the attack for every login action URL observed.
 * The idea behind the CWE_643 that is about XPath is similar to the XQuery injection in CWE_652.
 * XQuery supports XPath and extended relational models. According to the real cases, we can
 * trigger the weakness with the same inputs, then we merged these two CWE vulnerabilities
 * together.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1)
 * (2) Store the parameters of the current action in a variable
 * (3) It checks if the current action of the Input(1) is login.
 * (4) Filters the dynamic content.
 * (5) Verifies that the URL has not been tried before.
 * (6) Creates Input(2) as the follow-up input.
 * (7) Set a Xquery string for the username of the Input(2)
 * (8) Set a weak password value for the password of the Input(2)
 * (9) The system should show an error page
 *      Or the user should retrieve the content that has right to access it.
 */

```

```

MR CWE_643_652 {
{
  for ( Action action : Input(1).actions()){                                //(1)
    var pos = action.getPosition();                                         //(2)
    IMPLIES(
      isLogin(action) &&                                                    //(3)
      userCanRetrieveContent(action.getUser(), Output(Input(1),pos)) &&    //(4)
      notTried(action.url) &&                                              //(5)
      CREATE ( Input(2), Input(1)) &&                                     //(6)
      Input(2).actions.get(action.position).setValueForParametersOfType("text",
        XQueryInjection()) &&                                             //(7)
      Input(2).actions.get(action.position).setValueForParametersOfType("password",
        WeakPassword())                                                  //(8)
    ,
    OR(                                                                      //(9)
      isError(Output(Input(2), pos)),
      userCanRetrieveContent(action.getUser(), Output(Input(2),pos)))

    );//end-IMPLIES
  }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * The software receives input from an upstream component, but it does not handle or incorrectly
 * handles when an expected special element is missing.
 *
 * CWE_703:
 * The software does not properly anticipate or handle exceptional conditions that rarely occur
 * during normal operation of the software.
 *
 * CWE_166:
 * The software receives input from an upstream component, but it does not handle or incorrectly
 * handles when an expected special element is missing.
 *
 * CWE_78:
 * The software constructs all or part of an OS command using externally-influenced input from an
 * upstream component, but it does not neutralize or incorrectly neutralizes special elements
 * that could modify the intended OS command when it is sent to a downstream component.
 *
 * CWE_77:
 * The software constructs all or part of a command using externally-influenced input from an
 * upstream component, but it does not neutralize or incorrectly neutralizes special elements
 * that could modify the intended command when it is sent to a downstream component.
 *
 * CWE_76:
 * The software properly neutralizes certain special elements, but it improperly neutralizes
 * equivalent special elements.
 *
 * CWE_75:
 * The software does not adequately filter user-controlled input for special elements with
 * control implications.
 *
 * CWE_74:
 * The software constructs all or part of a command, data structure, or record using externally-
 * influenced input from an upstream component, but it does not neutralize or incorrectly
 * neutralize special elements that could modify how it is parsed or interpreted when it is sent
 * to a downstream component.
 *
 * CWE_91:
 * The software does not properly neutralize special elements that are used in XML, allowing
 * attackers to modify the syntax, content, or commands of the XML before it is processed by an
 * end system.
 *
 * CWE_790:
 * The software receives data from an upstream component, but does not filter or incorrectly
 * filters special elements before sending it to a downstream component.
 *
 * CWE_791:
 * The software receives data from an upstream component, but does not completely filter special
 * elements before sending it to a downstream component.

```

```

*
*
* Extra Details:
* Developers should anticipate that special elements will be removed in the input vectors of
* their software system. Use an appropriate combination of denylists and allowlists to ensure
* only valid, expected and appropriate input is processed by the system.
*
* Idea:
* As the input value it gives all possible special characters in the list to check that the
* system is vulnerable to them or not.
*
* Comments:
* (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
*     clicking on OK.
* (2) For loop iterates over all actions of the Input(1).
* (3) Iterates over the parameters of the action, which contains a form input.
* (4) Stores the parameters of the action in the variable.
* (5) Verifies that the user submits the form.
* (6) Checks if the URL has not seen before, to speed up the process.
* (7) Creates the follow-up input by copying the Input(1).
* (8) Sets the new input value which contains special character for the follow-up input.
* (9) The system should show an error page to the user or neutralize the effect of the special
*     character.
*
*
*/

```

```

MR CWE_703_166_78_77_76_75_74_91_790_791_B {
{
  keepDialogsOpen = true; // (1)
  for (Action action : Input(1).actions()){ // (2)
    for (var x = 0; action.containsFormInput() && x < action.formInputs.size; x++){ // (3)
      var pos = action.getPosition(); // (4)
      IMPLIES(
        action.isClickOnButton && // (5)
        notTried( x+action.url, Input(1).actions().get(pos).getElementURL() ) && // (6)
        CREATE(Input(2), Input(1) ) && // (7)
        Input(2).actions.get(pos).setFormInput(x, SpecialCharacters()) // (8)
      ,
      OR( // (9)
        isError(Output(Input(2), pos)),
        userCanRetrieveContent(action.getUser(), Output(Input(2),pos)))
      ); //end-IMPLIES
    } //end-for
  } //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 *  * CWE Definition:
 *  * The software receives input from an upstream component, but it does not handle or incorrectly
 *  * handles when an expected special element is missing.
 *
 *  * CWE_703:
 *  * The software does not properly anticipate or handle exceptional conditions that rarely occur
 *  * during normal operation of the software.
 *
 *  * CWE_166:
 *  * The software receives input from an upstream component, but it does not handle or incorrectly
 *  * handles when an expected special element is missing.
 *
 *  * CWE_78:
 *  * The software constructs all or part of an OS command using externally-influenced input from an
 *  * upstream component, but it does not neutralize or incorrectly neutralizes special elements
 *  * that could modify the intended OS command when it is sent to a downstream component.
 *
 *  * CWE_77:
 *  * The software constructs all or part of a command using externally-influenced input from an
 *  * upstream component, but it does not neutralize or incorrectly neutralizes special elements
 *  * that could modify the intended command when it is sent to a downstream component.
 *
 *  * CWE_76:
 *  * The software properly neutralizes certain special elements, but it improperly neutralizes
 *  * equivalent special elements.
 *
 *  * CWE_75:
 *  * The software does not adequately filter user-controlled input for special elements with
 *  * control implications.
 *
 *  * CWE_74:
 *  * The software constructs all or part of a command, data structure, or record using externally-
 *  * influenced input from an upstream component, but it does not neutralize or incorrectly
 *  * neutralize special elements that could modify how it is parsed or interpreted when it is sent
 *  * to a downstream component.
 *
 *  * CWE_91:
 *  * The software does not properly neutralize special elements that are used in XML, allowing
 *  * attackers to modify the syntax, content, or commands of the XML before it is processed by an
 *  * end system.
 *
 *  * CWE_790:
 *  * The software receives data from an upstream component, but does not filter or incorrectly
 *  * filters special elements before sending it to a downstream component.

```

```

*
* CWE_791:
* The software receives data from an upstream component, but does not completely filter special
* elements before sending it to a downstream component.
*
*
* Extra Details:
* Developers should anticipate that special elements will be removed in the input vectors of
* their software system. Use an appropriate combination of denylists and allowlists to ensure
* only valid, expected and appropriate input is processed by the system.
*
* Idea:
* As the input value it gives all possible special characters in the list to check that the
* system is vulnerable to them or not.
*
* Comments:
* (1) For loop iterates over all actions of the Input(1).
* (2) Iterates over all parameters of each action
* (3) Stores the parameters of the action in the variable.
* (4) Reads the user's input value and keeps it in a variable.
* (5) Checks if the URL was not tried before, to speed up the process.
* (6) Filters out the boolean input values.
* (7) Creates the follow-up input by copying the Input(1)
* (8) Sets the new input value which contains special character for the follow-up input.
* (9) The system should show an error page to the user or neutralize the effect of the special
*     character.
*
*/

MR CWE_703_166_78_77_76_75_74_91_790_791 {
{
  for ( Action action : Input(1).actions() ) {                                     //(1)
    for ( var par=0; par < action.getParameters().size(); par++ ){               //(2)
      var pos = action.getPosition();                                           //(3)
      var value = Input(1).actions().get(pos).getParameterValue(par);          //(4)
      IMPLIES(
        notTried( Input(1).actions().get(pos).getUrl() ) &&                    //(5)
        value != Boolean &&                                                     //(6)
        CREATE ( Input(2), Input(1) ) &&                                       //(7)
        Input(2).actions().get(pos).setParameterValue(par, SpecialCharacters()) //(8)
      ,
      OR(                                                                        //(9)
        isError(Output(Input(2), pos)),
        userCanRetrieveContent(action.getUser(), Output(Input(2),pos)))
      );//end-IMPLIES
    }//end-for
  }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import sun.security.util.Password

package smrl.mr.owasp {

/**
 * CWE Definition:
 * A protocol or its implementation supports interaction between multiple actors and allows those
 * actors to negotiate which algorithm should be used as a protection mechanism such as
 * encryption or authentication, but it does not select the strongest algorithm that is available
 * to both parties.
 *
 * Idea:
 * we need to configure the proxy then start collecting the information.
 * then inspect the collected information that looks for the client to change the algorithm to
 * the weak cipher, and check if we still can retrieve information from the server or not. look
 * for all pages.
 *
 * Comments:
 * (1) The loop iterates over all the actions of the input.
 * (2) checks that the URL of the action has not seen before.
 * (3) Checks if the action works on the encrypted channel.
 * (4) Defines a follow-up input.
 * (5) Sets the encryption algorithms to a weak one.
 * (6) Checks that the output generated by the action using the weak encryption algorithm lead to
 * different results.
 */

MR CWE_757_OTG_CrypST_004_F {
{
    for ( Action action : Input(1).actions()){                                //(1)
        IMPLIES (
            notTried(action.url) &&                                           //(2)
            isEncrypted( action ) &&                                           //(3)
            CREATE ( Input(2) , Input(1) ) &&                                  //(4)
            Input(2).setUseWeakEncryption( true )                             //(5)
        ,
            different(Output(Input(1), action.position), Output(Input(2), action.position)) //(6)
        );//end-IMPLIES
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_792: The software receives data from an upstream component, but does not completely
 * filter one or more instances of special elements before sending it to a downstream component.
 *
 * CWE_793: The software receives data from an upstream component, but only filters a single
 * instance of a special element before sending it to a downstream component.
 *
 * CWE_794: The software receives data from an upstream component, but does not filter all
 * instances of a special element before sending it to a downstream component.
 *
 * CWE_795: The software receives data from an upstream component, but only accounts for special
 * elements at a specified location, thereby missing remaining special elements that may exist
 * before sending it to a downstream component
 *
 * CWE_796: The software receives data from an upstream component, but only accounts for special
 * elements positioned relative to a marker (e.g. "at the beginning/end of a string; the second
 * argument"), thereby missing remaining special elements that may exist before sending it to a
 * downstream component.
 *
 * CWE_797: The software receives data from an upstream component, but only accounts for special
 * elements at an absolute position (e.g. "byte number 10"), thereby missing remaining special
 * elements that may exist before sending it to a downstream component.
 *
 * Idea:
 * In this MR, we will check if the system is vulnerable to special character injection or not.
 * In the other word, we test if the system under test has an input validation mechanism for the
 * inputs from the user.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogues are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Iterates over all parameters of each action
 * (4) Stores the parameters of the action in the variable.
 * (5) Reads the user's input value and keeps it in a variable.
 * (6) Checks if the URL was not tried before, to speed up the process.
 * (7) Filters out the boolean input values.
 * (8) Creates the follow-up input by copying the Input(1)
 * (9) Sets the new input value which contains special character for the follow-up input.
 * (10) Verifies that the system should show an error page or it would neutralize the input.
 *
 */
}

```

```

MR CWE_792_793_794_795_796_797_A{
{
    keepDialogsOpen = true; // (1)
    for ( Action action : Input(1).actions()){ // (2)
        for ( var par=0; par < action.getParameters().size(); par++ ){ // (3)
            var pos = action.getPosition(); // (4)
            var value = Input(1).actions().get(pos).getParameterValue(par); // (5)
            IMPLIES(
                notTried( Input(1).actions().get(pos).getUrl()) && // (6)
                value != Boolean && // (7)
                CREATE(Input(2), Input(1))&& // (8)
                Input(2).actions().get(pos).setParameterValue(par, SCInjection_beginning(
                    value , SpecialCharacters())) // (9)
            ,
            OR( // (10)
                isError(Output(Input(2), pos)),
                EQUAL(Output(Input(1), pos),Output(Input(2), pos)))
            );//end-IMPLIES
        }//end-for
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_792: The software receives data from an upstream component, but does not completely
 * filter one or more instances of special elements before sending it to a downstream component.
 *
 * CWE_793: The software receives data from an upstream component, but only filters a single
 * instance of a special element before sending it to a downstream component.
 *
 * CWE_794: The software receives data from an upstream component, but does not filter all
 * instances of a special element before sending it to a downstream component.
 *
 * CWE_795: The software receives data from an upstream component, but only accounts for special
 * elements at a specified location, thereby missing remaining special elements that may exist
 * before sending it to a downstream component
 *
 * CWE_796: The software receives data from an upstream component, but only accounts for special
 * elements positioned relative to a marker (e.g. "at the beginning/end of a string; the second
 * argument"), thereby missing remaining special elements that may exist before sending it to a
 * downstream component.
 *
 * CWE_797: The software receives data from an upstream component, but only accounts for special
 * elements at an absolute position (e.g. "byte number 10"), thereby missing remaining special
 * elements that may exist before sending it to a downstream component.
 *
 * Idea:
 * In this MR, we will check if the system is vulnerable to special character injection or not.
 * In the other word, we test if the system under test has an input validation mechanism for the
 * inputs from the user.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Iterates over all actions that contain a form input.
 * (4) Stores the parameters of the action in the variable.
 * (5) Reads the user's input value and keeps it in a variable.
 * (6) Checks if the user submits the form (action contains clicking on the submit button).
 * (7) Filters out the boolean input values.
 * (8) Checks if the URL was not tried before, to speed up the process.
 * (9) Creates the follow-up input by copying the Input(1)
 * (10) Sets the new input value which contains special character for the follow-up input.
 * (11) Verifies that the system should show an error page or it would neutralize the input.
 *
 */
}

```

```

MR CWE_792_793_794_795_796_797_B{
{
    keepDialogsOpen = true;                                     //(1)
    for (Action action : Input(1).actions()){                    //(2)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ //(3)
            var pos = action.getPosition();                      //(4)
            var value1 = Input(1).actions().get(pos).getFormInputs().get(x).getAsJsonObject(); //(5)
            IMPLIES(
                action.isClickOnButton &&                        //(6)
                value1 != Boolean &&                             //(7)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL()) && //(8)
                CREATE(Input(2), Input(1)) &&                   //(9)
                Input(2).actions.get(pos).setFormInput(x, SCInjection_beginning(value1 ,
                    SpecialCharacters()))                        //(10)
            ),
            OR(                                                  //(11)
                isError(Output(Input(2), pos)),
                EQUAL(Output(Input(1), pos),Output(Input(2), pos)))
            );//end-IMPLIES
        }//end-for
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_792: The software receives data from an upstream component, but does not completely
 * filter one or more instances of special elements before sending it to a downstream component.
 *
 * CWE_793: The software receives data from an upstream component, but only filters a single
 * instance of a special element before sending it to a downstream component.
 *
 * CWE_794: The software receives data from an upstream component, but does not filter all
 * instances of a special element before sending it to a downstream component.
 *
 * CWE_795: The software receives data from an upstream component, but only accounts for special
 * elements at a specified location, thereby missing remaining special elements that may exist
 * before sending it to a downstream component
 *
 * CWE_796: The software receives data from an upstream component, but only accounts for special
 * elements positioned relative to a marker (e.g. "at the beginning/end of a string; the second
 * argument"), thereby missing remaining special elements that may exist before sending it to a
 * downstream component.
 *
 * CWE_797: The software receives data from an upstream component, but only accounts for special
 * elements at an absolute position (e.g. "byte number 10"), thereby missing remaining special
 * elements that may exist before sending it to a downstream component.
 *
 * Idea:
 * In this MR, we will check if the system is vulnerable to special character injection or not.
 * In the other word, we test if the system under test has an input validation mechanism for the
 * inputs from the user.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Iterates over all parameters of each action
 * (4) Stores the parameters of the action in the variable.
 * (5) Reads the user's input value and keeps it in a variable.
 * (6) Checks if the URL was not tried before, to speed up the process.
 * (7) Filters out the boolean input values.
 * (8) Creates the follow-up input by copying the Input(1)
 * (9) Sets the new input value which contains special character for the follow-up input.
 * (10) Verifies that the system should show an error page or it would neutralize the input.
 *
 */
}

```

```

MR CWE_792_793_794_795_796_797_C{
{
    keepDialogsOpen = true; // (1)
    for ( Action action : Input(1).actions()){ // (2)
        for ( var par=0; par < action.getParameters().size(); par++ ){ // (3)
            var pos = action.getPosition(); // (4)
            var value = Input(1).actions().get(pos).getParameterValue(par); // (5)
            IMPLIES(
                notTried( Input(1).actions().get(pos).getUrl()) && // (6)
                value != Boolean && // (7)
                CREATE(Input(2), Input(1))&& // (8)
                Input(2).actions().get(pos).setParameterValue(par, SCInjection_Last(
                    value , SpecialCharacters())) // (9)
            ,
            OR( // (10)
                isError(Output(Input(2), pos)),
                EQUAL(Output(Input(1), pos),Output(Input(2), pos)))
            );//end-IMPLIES
        }//end-for
    }//end-for
}
} //end-MR
} // end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_792: The software receives data from an upstream component, but does not completely
 * filter one or more instances of special elements before sending it to a downstream component.
 *
 * CWE_793: The software receives data from an upstream component, but only filters a single
 * instance of a special element before sending it to a downstream component.
 *
 * CWE_794: The software receives data from an upstream component, but does not filter all
 * instances of a special element before sending it to a downstream component.
 *
 * CWE_795: The software receives data from an upstream component, but only accounts for special
 * elements at a specified location, thereby missing remaining special elements that may exist
 * before sending it to a downstream component
 *
 * CWE_796: The software receives data from an upstream component, but only accounts for special
 * elements positioned relative to a marker (e.g. "at the beginning/end of a string; the second
 * argument"), thereby missing remaining special elements that may exist before sending it to a
 * downstream component.
 *
 * CWE_797: The software receives data from an upstream component, but only accounts for special
 * elements at an absolute position (e.g. "byte number 10"), thereby missing remaining special
 * elements that may exist before sending it to a downstream component.
 *
 * Idea:
 * In this MR, we will check if the system is vulnerable to special character injection or not.
 * In the other word, we test if the system under test has an input validation mechanism for the
 * inputs from the user.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Iterates over all actions that contain a form input.
 * (4) Stores the parameters of the action in the variable.
 * (5) Reads the user's input value and keeps it in a variable.
 * (6) Checks if the user submits the form (action contains clicking on the submit button).
 * (7) Filters out the boolean input values.
 * (8) Checks if the URL was not tried before, to speed up the process.
 * (9) Creates the follow-up input by copying the Input(1)
 * (10) Sets the new input value which contains special character for the follow-up input.
 * (11) Verifies that the system should show an error page or it would neutralize the input.
 */
}

```

```

MR CWE_792_793_794_795_796_797_D{
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (3)
            var pos = action.getPosition(); // (4)
            var value1 = Input(1).actions().get(pos).getFormInputs().get(x).getAsJsonObject(); // (5)
            IMPLIES(
                action.isClickOnButton && // (6)
                value1 != Boolean && // (7)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL()) && // (8)
                CREATE(Input(2), Input(1)) && // (9)
                Input(2).actions.get(pos).setFormInput(x, SCInjection_Last(value1 ,
                    SpecialCharacters())) // (10)
            ,
            OR( // (11)
                isError(Output(Input(2), pos)),
                EQUAL(Output(Input(1), pos),Output(Input(2), pos)))
            ); //end-IMPLIES
        } //end-for
    } //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_792: The software receives data from an upstream component, but does not completely
 * filter one or more instances of special elements before sending it to a downstream component.
 *
 * CWE_793: The software receives data from an upstream component, but only filters a single
 * instance of a special element before sending it to a downstream component.
 *
 * CWE_794: The software receives data from an upstream component, but does not filter all
 * instances of a special element before sending it to a downstream component.
 *
 * CWE_795: The software receives data from an upstream component, but only accounts for special
 * elements at a specified location, thereby missing remaining special elements that may exist
 * before sending it to a downstream component
 *
 * CWE_796: The software receives data from an upstream component, but only accounts for special
 * elements positioned relative to a marker (e.g. "at the beginning/end of a string; the second
 * argument"), thereby missing remaining special elements that may exist before sending it to a
 * downstream component.
 *
 * CWE_797: The software receives data from an upstream component, but only accounts for special
 * elements at an absolute position (e.g. "byte number 10"), thereby missing remaining special
 * elements that may exist before sending it to a downstream component.
 *
 * Idea:
 * In this MR, we will check if the system is vulnerable to special character injection or not.
 * In the other word, we test if the system under test has an input validation mechanism for the
 * inputs from the user.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Iterates over all parameters of each action
 * (4) Stores the parameters of the action in the variable.
 * (5) Reads the user's input value and keeps it in a variable.
 * (6) Checks if the URL was not tried before, to speed up the process.
 * (7) Filters out the boolean input values.
 * (8) Creates the follow-up input by copying the Input(1)
 * (9) Sets the new input value which contains special character for the follow-up input.
 * (10) Verifies that the system should show an error page or it would neutralize the input.
 *
 */
}

```

```

MR CWE_792_793_794_795_796_797_E{
{
    keepDialogsOpen = true; // (1)
    for ( Action action : Input(1).actions()){ // (2)
        for ( var par=0; par < action.getParameters().size(); par++ ){ // (3)
            var pos = action.getPosition(); // (4)
            var value = Input(1).actions().get(pos).getParameterValue(par); // (5)
            IMPLIES(
                notTried( Input(1).actions().get(pos).getUrl()) && // (6)
                value != Boolean && // (7)
                CREATE(Input(2), Input(1))&& // (8)
                Input(2).actions().get(pos).setParameterValue(par, SCInjection_sides(
                    value , SpecialCharacters())) // (9)
            ,
            OR( // (10)
                isError(Output(Input(2), pos)),
                EQUAL(Output(Input(1), pos),Output(Input(2), pos)))
            );//end-IMPLIES
        }//end-for
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_792: The software receives data from an upstream component, but does not completely
 * filter one or more instances of special elements before sending it to a downstream component.
 *
 * CWE_793: The software receives data from an upstream component, but only filters a single
 * instance of a special element before sending it to a downstream component.
 *
 * CWE_794: The software receives data from an upstream component, but does not filter all
 * instances of a special element before sending it to a downstream component.
 *
 * CWE_795: The software receives data from an upstream component, but only accounts for special
 * elements at a specified location, thereby missing remaining special elements that may exist
 * before sending it to a downstream component
 *
 * CWE_796: The software receives data from an upstream component, but only accounts for special
 * elements positioned relative to a marker (e.g. "at the beginning/end of a string; the second
 * argument"), thereby missing remaining special elements that may exist before sending it to a
 * downstream component.
 *
 * CWE_797: The software receives data from an upstream component, but only accounts for special
 * elements at an absolute position (e.g. "byte number 10"), thereby missing remaining special
 * elements that may exist before sending it to a downstream component.
 *
 * Idea:
 * In this MR, we will check if the system is vulnerable to special character injection or not.
 * In the other word, we test if the system under test has an input validation mechanism for the
 * inputs from the user.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Iterates over all actions that contain a form input.
 * (4) Stores the parameters of the action in the variable.
 * (5) Reads the user's input value and keeps it in a variable.
 * (6) Checks if the user submits the form (action contains clicking on the submit button).
 * (7) Filters out the boolean input values.
 * (8) Checks if the URL was not tried before, to speed up the process.
 * (9) Creates the follow-up input by copying the Input(1)
 * (10) Sets the new input value which contains special character for the follow-up input.
 * (11) Verifies that the system should show an error page or it would neutralize the input.
 *
 */
}

```

```

MR CWE_792_793_794_795_796_797_F{
{
    keepDialogsOpen = true;                                     //(1)
    for (Action action : Input(1).actions()){                     //(2)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ //(3)
            var pos = action.getPosition();                       //(4)
            var value1 = Input(1).actions().get(pos).getFormInputs().get(x).getAsJsonObject(); //(5)
            IMPLIES(
                action.isClickOnButton &&                         //(6)
                value1 != Boolean &&                             //(7)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL()) && //(8)
                CREATE(Input(2), Input(1)) &&                   //(9)
                Input(2).actions.get(pos).setFormInput(x, SCInjection_sides(value1 ,
                    SpecialCharacters()))                        //(10)
            ,
            OR(                                                  //(11)
                isError(Output(Input(2), pos)),
                EQUAL(Output(Input(1), pos),Output(Input(2), pos)))
            );//end-IMPLIES
        }//end-for
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_792: The software receives data from an upstream component, but does not completely
 * filter one or more instances of special elements before sending it to a downstream component.
 *
 * CWE_793: The software receives data from an upstream component, but only filters a single
 * instance of a special element before sending it to a downstream component.
 *
 * CWE_794: The software receives data from an upstream component, but does not filter all
 * instances of a special element before sending it to a downstream component.
 *
 * CWE_795: The software receives data from an upstream component, but only accounts for special
 * elements at a specified location, thereby missing remaining special elements that may exist
 * before sending it to a downstream component
 *
 * CWE_796: The software receives data from an upstream component, but only accounts for special
 * elements positioned relative to a marker (e.g. "at the beginning/end of a string; the second
 * argument"), thereby missing remaining special elements that may exist before sending it to a
 * downstream component.
 *
 * CWE_797: The software receives data from an upstream component, but only accounts for special
 * elements at an absolute position (e.g. "byte number 10"), thereby missing remaining special
 * elements that may exist before sending it to a downstream component.
 *
 * Idea:
 * In this MR, we will check if the system is vulnerable to special character injection or not.
 * In the other word, we test if the system under test has an input validation mechanism for the
 * inputs from the user.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Iterates over all parameters of each action
 * (4) Stores the parameters of the action in the variable.
 * (5) Reads the user's input value and keeps it in a variable.
 * (6) Checks if the URL was not tried before, to speed up the process.
 * (7) Filters out the boolean input values.
 * (8) Creates the follow-up input by copying the Input(1)
 * (9) Sets the new input value which contains special character for the follow-up input.
 * (10) Verifies that the system should show an error page or it would neutralize the input.
 *
 */
}

```

```

MR CWE_792_793_794_795_796_797_G{
{
    keepDialogsOpen = true; // (1)
    for ( Action action : Input(1).actions()){ // (2)
        for ( var par=0; par < action.getParameters().size(); par++ ){ // (3)
            var pos = action.getPosition(); // (4)
            var value = Input(1).actions().get(pos).getParameterValue(par); // (5)
            IMPLIES(
                notTried( Input(1).actions().get(pos).getUrl()) && // (6)
                value != Boolean && // (7)
                CREATE(Input(2), Input(1))&& // (8)
                Input(2).actions().get(pos).setParameterValue(par, SCInjection_beginning_double(
                    value , SpecialCharacters())) // (9)
            ,
            OR( // (10)
                isError(Output(Input(2), pos)),
                EQUAL(Output(Input(1), pos),Output(Input(2), pos)))
            );//end-IMPLIES
        }//end-for
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * * CWE Definition:
 * * CWE_792: The software receives data from an upstream component, but does not completely
 * * filter one or more instances of special elements before sending it to a downstream component.
 * *
 * * CWE_793: The software receives data from an upstream component, but only filters a single
 * * instance of a special element before sending it to a downstream component.
 * *
 * * CWE_794: The software receives data from an upstream component, but does not filter all
 * * instances of a special element before sending it to a downstream component.
 * *
 * * CWE_795: The software receives data from an upstream component, but only accounts for special
 * * elements at a specified location, thereby missing remaining special elements that may exist
 * * before sending it to a downstream component
 * *
 * * CWE_796: The software receives data from an upstream component, but only accounts for special
 * * elements positioned relative to a marker (e.g. "at the beginning/end of a string; the second
 * * argument"), thereby missing remaining special elements that may exist before sending it to a
 * * downstream component.
 * *
 * * CWE_797: The software receives data from an upstream component, but only accounts for special
 * * elements at an absolute position (e.g. "byte number 10"), thereby missing remaining special
 * * elements that may exist before sending it to a downstream component.
 * *
 * * Idea:
 * * In this MR, we will check if the system is vulnerable to special character injection or not.
 * * In the other word, we test if the system under test has an input validation mechanism for the
 * * inputs from the user.
 * *
 * * Comments:
 * * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * * clicking on OK.
 * * (2) For loop iterates over all actions of the Input(1).
 * * (3) Iterates over all actions that contain a form input.
 * * (4) Stores the parameters of the action in the variable.
 * * (5) Reads the user's input value and keeps it in a variable.
 * * (6) Checks if the user submits the form (action contains clicking on the submit button).
 * * (7) Filters out the boolean input values.
 * * (8) Checks if the URL was not tried before, to speed up the process.
 * * (9) Creates the follow-up input by copying the Input(1)
 * * (10) Sets the new input value which contains special character for the follow-up input.
 * * (11) Verifies that the system should show an error page or it would neutralize the input.
 * *
 * */

```

```

MR CWE_792_793_794_795_796_797_H{
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (3)
            var pos = action.getPosition(); // (4)
            var value1 = Input(1).actions().get(pos).getFormInputs().get(x).getAsJsonObject(); // (5)
            IMPLIES(
                action.isClickOnButton && // (6)
                value1 != Boolean && // (7)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL()) && // (8)
                CREATE(Input(2), Input(1)) && // (9)
                Input(2).actions.get(pos).setFormInput(x, SCInjection_beginning_double(value1 ,
                    SpecialCharacters())) // (10)
            ,
            OR( // (11)
                isError(Output(Input(2), pos)),
                EQUAL(Output(Input(1), pos),Output(Input(2), pos)))
            ); //end-IMPLIES
        } //end-for
    } //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 *
 * CWE Definition:
 * CWE_792: The software receives data from an upstream component, but does not completely
 * filter one or more instances of special elements before sending it to a downstream component.
 *
 * CWE_793: The software receives data from an upstream component, but only filters a single
 * instance of a special element before sending it to a downstream component.
 *
 * CWE_794: The software receives data from an upstream component, but does not filter all
 * instances of a special element before sending it to a downstream component.
 *
 * CWE_795: The software receives data from an upstream component, but only accounts for special
 * elements at a specified location, thereby missing remaining special elements that may exist
 * before sending it to a downstream component
 *
 * CWE_796: The software receives data from an upstream component, but only accounts for special
 * elements positioned relative to a marker (e.g. "at the beginning/end of a string; the second
 * argument"), thereby missing remaining special elements that may exist before sending it to a
 * downstream component.
 *
 * CWE_797: The software receives data from an upstream component, but only accounts for special
 * elements at an absolute position (e.g. "byte number 10"), thereby missing remaining special
 * elements that may exist before sending it to a downstream component.
 *
 * Idea:
 * In this MR, we will check if the system is vulnerable to special character injection or not.
 * In the other word, we test if the system under test has an input validation mechanism for the
 * inputs from the user.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Iterates over all parameters of each action
 * (4) Stores the parameters of the action in the variable.
 * (5) Reads the user's input value and keeps it in a variable.
 * (6) Checks if the URL was not tried before, to speed up the process.
 * (7) Filters out the boolean input values.
 * (8) Creates the follow-up input by copying the Input(1)
 * (9) Sets the new input value which contains special character for the follow-up input.
 * (10) Verifies that the system should show an error page or it would neutralize the input.
 *
 */
}

```

```

MR CWE_792_793_794_795_796_797_I{
{
    keepDialogsOpen = true;                                     //(1)
    for ( Action action : Input(1).actions()){                  //(2)
        for ( var par=0; par < action.getParameters().size(); par++ ){          //(3)
            var pos = action.getPosition();                      //(4)
            var value = Input(1).actions().get(pos).getParameterValue(par);      //(5)
            IMPLIES(
                notTried( Input(1).actions().get(pos).getUrl()) &&          //(6)
                value != Boolean &&                                       //(7)
                CREATE(Input(2), Input(1))&&                          //(8)
                Input(2).actions().get(pos).setParameterValue(par, SCInjection_Last_double(
                    value , SpecialCharacters()))                      //(9)
            ,
            OR(                                                         //(10)
                isError(Output(Input(2), pos)),
                EQUAL(Output(Input(1), pos),Output(Input(2), pos)))
            );//end-IMPLIES
        }//end-for
    }//end-for
}
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_792: The software receives data from an upstream component, but does not completely
 * filter one or more instances of special elements before sending it to a downstream component.
 *
 * CWE_793: The software receives data from an upstream component, but only filters a single
 * instance of a special element before sending it to a downstream component.
 *
 * CWE_794: The software receives data from an upstream component, but does not filter all
 * instances of a special element before sending it to a downstream component.
 *
 * CWE_795: The software receives data from an upstream component, but only accounts for special
 * elements at a specified location, thereby missing remaining special elements that may exist
 * before sending it to a downstream component
 *
 * CWE_796: The software receives data from an upstream component, but only accounts for special
 * elements positioned relative to a marker (e.g. "at the beginning/end of a string; the second
 * argument"), thereby missing remaining special elements that may exist before sending it to a
 * downstream component.
 *
 * CWE_797: The software receives data from an upstream component, but only accounts for special
 * elements at an absolute position (e.g. "byte number 10"), thereby missing remaining special
 * elements that may exist before sending it to a downstream component.
 *
 * Idea:
 * In this MR, we will check if the system is vulnerable to special character injection or not.
 * In the other word, we test if the system under test has an input validation mechanism for the
 * inputs from the user.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Iterates over all actions that contain a form input.
 * (4) Stores the parameters of the action in the variable.
 * (5) Reads the user's input value and keeps it in a variable.
 * (6) Checks if the user submits the form (action contains clicking on the submit button).
 * (7) Filters out the boolean input values.
 * (8) Checks if the URL was not tried before, to speed up the process.
 * (9) Creates the follow-up input by copying the Input(1)
 * (10) Sets the new input value which contains special character for the follow-up input.
 * (11) Verifies that the system should show an error page or it would neutralize the input.
 *
 */
}

```

```

MR CWE_792_793_794_795_796_797_J{
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (3)
            var pos = action.getPosition(); // (4)
            var value1 = Input(1).actions().get(pos).getFormInputs().get(x).getAsJsonObject(); // (5)
            IMPLIES(
                action.isClickOnButton && // (6)
                value1 != Boolean && // (7)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL()) && // (8)
                CREATE(Input(2), Input(1)) && // (9)
                Input(2).actions.get(pos).setFormInput(x, SCInjection_Last_double(value1 ,
                    SpecialCharacters())) // (10)
            ,
            OR( // (11)
                isError(Output(Input(2), pos)),
                EQUAL(Output(Input(1), pos),Output(Input(2), pos)))
            ); //end-IMPLIES
        } //end-for
    } //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_792: The software receives data from an upstream component, but does not completely
 * filter one or more instances of special elements before sending it to a downstream component.
 *
 * CWE_793: The software receives data from an upstream component, but only filters a single
 * instance of a special element before sending it to a downstream component.
 *
 * CWE_794: The software receives data from an upstream component, but does not filter all
 * instances of a special element before sending it to a downstream component.
 *
 * CWE_795: The software receives data from an upstream component, but only accounts for special
 * elements at a specified location, thereby missing remaining special elements that may exist
 * before sending it to a downstream component
 *
 * CWE_796: The software receives data from an upstream component, but only accounts for special
 * elements positioned relative to a marker (e.g. "at the beginning/end of a string; the second
 * argument"), thereby missing remaining special elements that may exist before sending it to a
 * downstream component.
 *
 * CWE_797: The software receives data from an upstream component, but only accounts for special
 * elements at an absolute position (e.g. "byte number 10"), thereby missing remaining special
 * elements that may exist before sending it to a downstream component.
 *
 * Idea:
 * In this MR, we will check if the system is vulnerable to special character injection or not.
 * In the other word, we test if the system under test has an input validation mechanism for the
 * inputs from the user.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Iterates over all parameters of each action
 * (4) Stores the parameters of the action in the variable.
 * (5) Reads the user's input value and keeps it in a variable.
 * (6) Checks if the URL was not tried before, to speed up the process.
 * (7) Filters out the boolean input values.
 * (8) Creates the follow-up input by copying the Input(1)
 * (9) Sets the new input value which contains special character for the follow-up input.
 * (10) Verifies that the system should show an error page or it would neutralize the input.
 *
 */
}

```

```

MR CWE_792_793_794_795_796_797_K{
{
    keepDialogsOpen = true; // (1)
    for ( Action action : Input(1).actions()){ // (2)
        for ( var par=0; par < action.getParameters().size(); par++ ){ // (3)
            var pos = action.getPosition(); // (4)
            var value = Input(1).actions().get(pos).getParameterValue(par); // (5)
            IMPLIES(
                notTried( Input(1).actions().get(pos).getUrl()) && // (6)
                value != Boolean && // (7)
                CREATE(Input(2), Input(1))&& // (8)
                Input(2).actions().get(pos).setParameterValue(par, SCInjection_middle(
                    value , SpecialCharacters())) // (9)
            ,
            OR( // (10)
                isError(Output(Input(2), pos)),
                EQUAL(Output(Input(1), pos),Output(Input(2), pos)))
            );//end-IMPLIES
        }//end-for
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_792: The software receives data from an upstream component, but does not completely
 * filter one or more instances of special elements before sending it to a downstream component.
 *
 * CWE_793: The software receives data from an upstream component, but only filters a single
 * instance of a special element before sending it to a downstream component.
 *
 * CWE_794: The software receives data from an upstream component, but does not filter all
 * instances of a special element before sending it to a downstream component.
 *
 * CWE_795: The software receives data from an upstream component, but only accounts for special
 * elements at a specified location, thereby missing remaining special elements that may exist
 * before sending it to a downstream component
 *
 * CWE_796: The software receives data from an upstream component, but only accounts for special
 * elements positioned relative to a marker (e.g. "at the beginning/end of a string; the second
 * argument"), thereby missing remaining special elements that may exist before sending it to a
 * downstream component.
 *
 * CWE_797: The software receives data from an upstream component, but only accounts for special
 * elements at an absolute position (e.g. "byte number 10"), thereby missing remaining special
 * elements that may exist before sending it to a downstream component.
 *
 * Idea:
 * In this MR, we will check if the system is vulnerable to special character injection or not.
 * In the other word, we test if the system under test has an input validation mechanism for the
 * inputs from the user.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Iterates over all actions that contain a form input.
 * (4) Stores the parameters of the action in the variable.
 * (5) Reads the user's input value and keeps it in a variable.
 * (6) Checks if the user submits the form (action contains clicking on the submit button).
 * (7) Filters out the boolean input values.
 * (8) Checks if the URL was not tried before, to speed up the process.
 * (9) Creates the follow-up input by copying the Input(1)
 * (10) Sets the new input value which contains special character for the follow-up input.
 * (11) Verifies that the system should show an error page or it would neutralize the input.
 *
 */
}

```

```

MR CWE_792_793_794_795_796_797_L{
{
    keepDialogsOpen = true;                                     //(1)
    for (Action action : Input(1).actions()){                    //(2)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ //(3)
            var pos = action.getPosition();                      //(4)
            var value1 = Input(1).actions().get(pos).getFormInputs().get(x).getAsJsonObject(); //(5)
            IMPLIES(
                action.isClickOnButton &&                        //(6)
                value1 != Boolean &&                             //(7)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL()) && //(8)
                CREATE(Input(2), Input(1)) &&                   //(9)
                Input(2).actions.get(pos).setFormInput(x, SCInjection_middle(value1 ,
                    SpecialCharacters()))                        //(10)
            ,
            OR(                                                  //(11)
                isError(Output(Input(2), pos)),
                EQUAL(Output(Input(1), pos),Output(Input(2), pos)))
            );//end-IMPLIES
        }//end-for
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import sun.security.util.Password

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * XSS: The software does not neutralize or incorrectly neutralizes user-controllable input
 * before it is placed in output that is used as a web page that is served to other users.
 *
 * Idea:
 * for all actions we inject the URL of the action with XSS strings to verify that
 * is there any vulnerable URL in the system or not. The system should show an error page
 * or a different output after injecting the URL.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) The second loop iterates over all parameters of the action.
 * (4) Defines a variable to store the position of the Input(1)'s action.
 * (5) Checks if the parameter of the action URL has not seen before, to speed up the process.
 * (6) Verifies that the action does not originally contain any alert.
 * (7) Creates the follow-up input by copying the Input(1).
 * (8) Sets the injected XSS string as the parameter value of the follow-up input.
 * (9) Checks either the attack was not performed or no effect shall be observed (the effect of
 * the XSS is usually visualized when reaching the page where we inject the XSS)
 */

MR CWE_79_a_XSSreflected {
{
    keepDialogsOpen = true; // (1)
    for ( Action action : Input(1).actions() ) { // (2)
        for ( var x = 0; x < action.parameters.size; x++){ // (3)
            var pos = action.getPosition(); // (4)
            IMPLIES(
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL() ) && // (5)
                !Output(Input(1), pos).hasAlert && // (6)
                CREATE( Input(2), Input(1) ) && // (7)
                Input(2).actions().get(pos).setParameterValue(x,XSSInjectionString()) // (8)
            ,
            OR( // (9)
                Output(Input(2), pos).emptyFile,
                !Output(Input(2), pos).hasAlert )
            ); // end-IMPLIES
        } // end-for
    } // end-for
}
} // end-MR
} // end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import smrl.mr.language.actions.StandardAction;
import jdk.internal.util.xml.impl.Input

package smrl.mr.owasp{

/**
 * CWE Definition:
 * The software does not neutralize or incorrectly neutralizes user-controllable input
 * before it is placed in output that is used as a web page that is served to other users.
 *
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1)
 * (3) Stores the parameters of the current action in a variable
 * (4) Iterates over the actions that contain a form input.
 * (5) Checks if the action submits the form; i.e. the action contains clicking on submit button.
 * (6) Verifies that the URL has not been tried before.
 * (7) Checks if no pop-up / alert dialog is shown normally.
 * (8) Creates Input(2) as the follow-up input.
 * (9) Set the XSS string as the form input value.
 * (10) Either the attack was not performed or there is no pop-up in the output (reflected XSS
 * immediately leads to a pop-up).
 */

MR CWE_79b_XSSreflected {
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        var pos = action.getPosition(); // (3)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (4)
            IMPLIES(
                action.isClickOnButton && // (5)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL() ) && // (6)
                !Output(Input(1),pos).hasAlert && // (7)
                CREATE(Input(2), Input(1) ) && // (8)
                Input(2).actions.get(pos).setFormInput(x, XSSInjectionString()) // (9)
            ,
            OR( // (10)
                Output(Input(2), pos).emptyFile,
                !Output(Input(2), pos).hasAlert
            ); // end-IMPLIES
        } // end-for
    } // end-for
}
} // end-MR
} // end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import smrl.mr.language.actions.StandardAction;

package smrl.mr.owasp{

/**
 * CWE Definition:
 * The software does not neutralize or incorrectly neutralizes user-controllable input
 * before it is placed in output that is used as a web page that is served to other users.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 *     clicking on OK.
 * (2) For loop iterates over all actions of the Input(1)
 * (3) Stores the parameters of the current action in a variable
 * (4) Iterates over the actions that contain a form input.
 * (5) Checks if the action submits the form; i.e. the action contains clicking on submit button.
 * (6) Verifies that the URL has not been tried before.
 * (7) Checks if no pop-up / alert dialog is shown normally.
 * (8) Creates Input(2) as the follow-up input.
 * (9) Set the XSS string as the form input value.
 * (10) We will perform the same sequence as the source-input (i.e., without XSS injected)
 *      If an alert will pop-up, it will mean that the XSS has been stored into the system
 * (11) Either the attack was not performed or no effect shall be observed (the effect of
 *      the XSS is usually visualized when reaching the page where we inject the XSS).
 */

MR CWE_79c_storedXSS {
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        var pos = action.getPosition(); // (3)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (4)
            IMPLIES(
                action.isClickOnButton && // (5)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL()) && // (6)
                !Output(Input(1),pos).hasAlert && // (7)
                CREATE(Input(2), Input(1)) && // (8)
                Input(2).actions.get(pos).setFormInput(x, XSSInjectionString()) && // (9)
                CREATE(Input(3), Input(1) ) // (10)
            ,
            OR( // (11)
                Output(Input(2), pos).emptyFile,
                !Output(Input(3), pos-1).hasAlert)
            ); // end-IMPLIES
        } // end-for
    } // end-for
}
} // end-MR
} // end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.actions.ResetSUTAction;

package smrl.mr.owasp{

/*
 * CWE Definition:
 * The software supports a session in which more than one behavior must be performed by an actor,
 * but it does not properly ensure that the actor performs the behaviors in the required
 * sequence.
 *
 * Idea:
 * reverse the sequence of an action that requires to be authenticated before take it.
 *
 * E.g. if we signup, the email should be verified, and the steps are like the following:
 *      signup -> email verification(enter the code) -> login -> authenticated action
 *      signup -> login-> authenticated action NOTVALID
 *
 *      Considering that we have a website that after signing up it redirect you to a
 *      page that asks you to enter a code,
 *      which has been sent to your email account (email verification).
 *      After that it will redirect you to login page.
 *
 * Comments:
 * (1) The first loop iterates over all the actions to find a signup action.
 * (2) The second loop iterates over all the actions to find a login action performed after the
 *      signup.
 * (3) Verifies that the action y is a login.
 * (4) Creates the follow-up input with the sequence of actions after the login (action y).
 * (5) Makes sure that we start with a signup action
 * (6) Reset the actions for next iteration
 * (7) Verifies that the system gives two different outputs or will give an error by executing
 *      the follow-up input.
 */

MR CWE_841 {
{
    for(var x = 0; x < Input(1).actions().size(); x++ ){                                //(1)
        for(var y = x+2; isSignup(Input(1).actions().get(x)) && (y < Input(1).actions().size());
            y++){                                                                    //(2)
            IMPLIES (
                isLogin(Input(1).actions().get(y))&&                                //(3)
                CREATE(Input(2),Input(Input(1).actions().subList(y, Input(1).actions().size()))&& //(4)
                Input(2).addAction(0,Input(1).actions().get(x)) &&                //(5)
                Input(2).addAction(0, new ResetSUTAction())                        //(6)
            ),
            ,

            AND(                                                                    //(7)
                different ( Output(Input(2),1), Output(Input(1),y)),
                isError(Output(Input(2),1)))
            ); //end-IMPLIES
        } //end-for
    } //end-for
}

```

```
}  
} //end-MR  
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 * CWE Definition:
 * The software constructs a string for a command to executed by a separate component in another
 * control sphere, but it does not properly delimit the intended arguments, options, or switches
 * within that command string.
 *
 * Idea:
 * For all parameters of the actions we try to inject some commands to check whether the system
 * is able to neutralize the command injection or not.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Stores the position of the current action in a variable.
 * (4) Stores the size of the action in a variable.
 * (5) Second for loop iterates over all parameters of the current action
 * (6) Verifies that the current URL has not been seen before
 * (7) Filters the webpages with dynamic content.
 * (8) Creates the follow-up input.
 * (9) Sets the command injection string to the parameter of the action
 * (10) Checks that the system does not parse the command and the Input(1) and the follow-up
 * input both get the same output.
 * (Assuming that the input sequence continues after 'pos', we may observe the effects of the
 * attack in the following inputs).
 * Or the result of the follow-up input is error
 * Or the follow-up input can has the right to access
 */

MR CWE_88a {
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        var pos = action.getPosition(); // (3)
        var size = Input (1).actions().size(); // (4)
        for (var x = 0; x < action.parameters.size; x++){ // (5)
            IMPLIES(
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL()) && // (6)
                userCanRetrieveContent(action.getUser(),Output(Input(1)).values
                    .subList(pos,size))&& // (7)
                CREATE(Input(2), Input(1))&& // (8)
                Input(2).actions().get(pos).setParameterValue( x, CommandInjection()) // (9)
            ),
            OR( // (10)
                !different(Output(Input(1)).values.subList(pos,Input(1).actions.Length),
                    Output(Input(2)).values.subList(pos,Input(2).actions.Length))
                , OR(
                    isError(Output(Input(2), pos)),

```

```
        userCanRetrieveContent(action.getUser(), Output(Input(2))
                               .values.subList(pos,size) )))
    };//end-IMPLIES
  }//end-for
}//end-for
}
}//end-MR
}//end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * The software constructs a string for a command to executed by a separate component in another
 * control sphere, but it does not properly delimit the intended arguments, options, or switches
 * within that command string.
 *
 * Idea:
 * For all parameters of the actions we try to inject some commands to check whether the system
 * is able to neutralize the command injection or not.
 *
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 *     clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Stores the position of the current action in a variable.
 * (4) Stores the size of the action in a variable.
 * (5) Second for loop iterates over all parameters of the action that contains form input.
 * (6) Checks if the user will submit the form.
 * (7) Verifies that the current URL has not been seen before.
 * (8) Filters the web pages with dynamic content.
 * (9) Creates the follow-up input
 * (10) Sets the command injection strings to the parameter of the action
 * (11) Checks that the system does not parse the command and the Input(1) and the follow-up
 *      input both get the same output.
 *      (Assuming that the input sequence continues after 'pos', we may observe the effects of the
 *      attack in the following inputs).
 *      Or the result of the follow-up input is error
 *      Or the follow-up input can has the right to access
 *
 */

MR CWE_88b {
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        var pos = action.getPosition(); // (3)
        var size = Input (1).actions().size(); // (4)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (5)
            IMPLIES(
                action.isClickOnButton && // (6)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL()) && // (7)
                userCanRetrieveContent(action.getUser(), Output(Input(1))
                    .values.subList(pos,size)) && // (8)
                CREATE(Input(2), Input(1)) && // (9)
                Input(2).actions.get(pos).setFormInput(x, CommandInjection()) // (10)
            ,

```

```

OR(                                                                                                     //(11)
    !different(Output(Input(1)).values.subList(pos,Input(1).actions.Length),
                Output(Input(2)).values.subList(pos,Input(2).actions.Length))
    ,
    OR(
        isError( Output(Input(2) , pos) ),
        userCanRetrieveContent(action.getUser(), Output(Input(2))
                                .values.subList(pos,size))))
    );//end-IMPLIES
} //end-for
} //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import smrl.mr.language.actions.StandardAction;
import jdk.internal.util.xml.impl.Input

package smrl.mr.owasp{

/**
 * CWE Definition:
 * CWE_89:
 * The software constructs all or part of an SQL command using externally-influenced input
 * from an upstream component, but it does not neutralize or incorrectly neutralizes special
 * elements that could modify the intended SQL command when it is sent to a downstream component.
 *
 * CWE_943:
 * The application generates a query intended to access or manipulate data in a data store such
 * as a database, but it does not neutralize or incorrectly neutralizes special elements that can
 * modify the intended logic of the query.
 *
 * Idea:
 * If the system creates a query that accesses the data in the data store and uses the user's
 * input, the system should neutralize the special elements of the user's input. CWE_89 and
 * CWE_943 emphasize the importance of neutralizing the user's input; then, we merged these two
 * weaknesses as they target the same weakness.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1)
 * (3) Stores the parameters of the current action in a variable
 * (4) Iterates over the actions that contain a form input.
 * (5) Checks if the action submits the form; i.e. the action contains clicking on submit button.
 * (6) Filters the dynamic content.
 * (7) Verifies that the URL has not been tried before.
 * (8) Creates Input(2) as the follow-up input.
 * (9) Set the SQL string as the form input value.
 * (10) The system should show an error page or a content that the user has right to access.
 */
MR CWE_89_943_b {
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        var pos = action.getPosition(); // (3)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (4)
            IMPLIES(
                action.isClickOnButton && // (5)
                userCanRetrieveContent(action.getUser(), Output(Input(1),pos))&& // (6)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL() ) && // (7)
                CREATE(Input(2), Input(1) ) && // (8)
                Input(2).actions.get(pos).setFormInput(x, SQLInjectionString()) // (9)
            ,
            OR( // (10)
                isError(Output(Input(2), pos)),
                userCanRetrieveContent(action.getUser(), Output(Input(2),pos)))
            )
        }
    }
}
}

```

```
        );//end-IMPLIES
    }//end-for
} //end-for
}
} //end-MR
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import smrl.mr.language.actions.StandardAction;
import jdk.internal.util.xml.impl.Input

package smrl.mr.owasp{

/**
 * CWE Definition:
 * CWE_89:
 * The software constructs all or part of an SQL command using externally-influenced input
 * from an upstream component, but it does not neutralize or incorrectly neutralizes special
 * elements that could modify the intended SQL command when it is sent to a downstream component.
 *
 * CWE_943:
 * The application generates a query intended to access or manipulate data in a data store such
 * as a database, but it does not neutralize or incorrectly neutralizes special elements that can
 * modify the intended logic of the query.
 *
 * Idea:
 * If the system creates a query that accesses the data in the data store and uses the user's
 * input, the system should neutralize the special elements of the user's input. CWE_89 and
 * CWE_943 emphasize the importance of neutralizing the user's input; then, we merged these two
 * weaknesses as they target the same weakness.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1)
 * (3) Stores the parameters of the current action in a variable
 * (4) Iterates over the actions that contain a login form input.
 * (5) Checks if the action submits the form; i.e. the action contains clicking on submit button.
 * (6) Filters the dynamic content.
 * (7) Verifies that the URL has not been tried before.
 * (8) Verifies the output of the action does not contain any errors.
 * (9) Creates Input(2) as the follow-up input.
 * (10) Set the SQL string as the form input value.
 * (11) The system should show an error page or a content that the user has right to access.
 */

MR CWE_89_943_C {
{
    keepDialogsOpen = true; // (1)
    for ( Action action : Input(1).actions()) { // (2)
        var pos = action.getPosition(); // (3)
        for (var x = 0; action.containLoginFormInput() && x < action.formInputs.size; x++){ // (4)
            IMPLIES(
                action.isClickOnButton && // (5)
                userCanRetrieveContent(action.getUser(), Output(Input(1),pos)) && // (6)
                notTried(x+action.url, Input(1).actions().get(pos).getElementURL()) && // (7)
                !isError(Output(Input(1),pos)) && // (8)
                CREATE ( Input(2), Input(1) ) && // (9)
                Input(2).actions.get(pos).setFormInput(x, SQLInjectionString()) // (10)
            ,

```

```
OR(                                                                    //(11)
    isError(Output(Input(2), pos)),
    userCanRetrieveContent(action.getUser(), Output(Input(2),pos)))
);//end-IMPLIES
} //end-for
} //end-for
}
} //end-MR
} //end-package
```

```

import static smrl.mr.language.Operations.*;

import smrl.mr.language.Action;

package smrl.mr.owasp{

/**
 * CWE_definition:
 * CWE_943:
 * The application generates a query intended to access or manipulate data in a data store such
 * as a database, but it does not neutralize or incorrectly neutralizes special elements that can
 * modify the intended logic of the query.
 *
 * CWE_89(Child of CWE_943):
 * The software constructs all or part of an SQL command using externally-influenced input
 * from an upstream component, but it does not neutralize or incorrectly neutralizes special
 * elements that could modify the intended SQL command when it is sent to a downstream component.
 *
 * Idea:
 * If the system creates a query that accesses the data in the data store and uses the user's
 * input, the system should neutralize the special elements of the user's input. CWE_89 and
 * CWE_943 emphasize the importance of neutralizing the user's input; then, we merged these two
 * weaknesses as they target the same weakness.
 *
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Stores the parameters of the current action in a variable.
 * (3) Second for loop iterates over all parameters of the current action.
 * (4) Verifies that the current URL has not been seen before, to speed up the process.
 * (5) Filters the dynamic content.
 * (6) Creates the follow-up input.
 * (7) Sets the SQL injection strings to the parameter of the action.
 * (8) Checks that the result of the follow-up input is error.
 *      Or the follow-up input can retrieve the content that has the right to access
 *
 */

MR CWE_89_943 {
{
    for (Action action : Input(1).actions()){                                //(1)
        var pos = action.getPosition();                                        //(2)
        for (var x = 0; x < action.parameters.size; x++){                    //(3)
            IMPLIES(
                notTried(x + action.url, Input(1).actions().get(pos).getElementURL()) && //(4)
                userCanRetrieveContent(action.getUser(), Output(Input(1),pos)) &&    //(5)
                CREATE(Input(2), Input(1))&&                                     //(6)
                Input(2).actions().get(pos).setParameterValue( x, SQLInjectionString()) //(7)
            ,
            OR(                                                                    //(8)
                isError(Output(Input(2), pos)),
                userCanRetrieveContent(action.getUser(), Output(Input(2),pos)))
            );//end-IMPLIES
        }//end-for
    }
}

```

```
    }//end-for  
  }  
} //end-MR  
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
package smrl.mr.owasp{

/**
 * CWE_definition:
 *
 * The software constructs all or part of an LDAP query using externally-influenced input from
 * an upstream component, but it does not neutralize or incorrectly neutralizes special elements
 * that could modify the intended LDAP query when it is sent to a downstream component.
 *
 * Idea:
 * We try to inject some LDAP query strings to all input entries of the actions, then
 * we will check whether the system can neutralize the injected inputs or not.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Second for loop iterates over all parameters of the current action.
 * (3) Stores the position of the action parameter in a variable.
 * (4) Stores the parameter name.
 * (5) Verifies that we did not already test the parameter for the current URL.
 * (6) Checks that the action does not originally contain any alert.
 * (7) Filters out the web pages with dynamic content.
 * (8) Creates the follow-up input
 * (9) Sets the LDAP query strings to the parameter of the action
 * (10) Checks that the result of the follow-up input is different than the source input
 *      Or The output contains an alert or an error page is shown
 *      Or the follow-up input can retrieve a content that has the right to access
 */

MR CWE_90a {
{
for (Action action : Input(1).actions()){
for (var x = 0; x < action.parameters.size; x++){
var pos = action.getPosition();
var parameterName = action.parameters.get(x).key;
IMPLIES(
    notTried(LDAPInjectionString()+ x + parameterName, Input(1).actions()
        .get(pos).getUrl()) && //(5)
    NOT(NULL(Output(Input(1), pos)))&& //(6)
    userCanRetrieveContent(action.getUser(), Output(Input(1),pos)) && //(7)
    CREATE(Input(2), Input(1))&& //(8)
    Input(2).actions().get(pos).setParameterValue( x, LDAPInjectionString()) //(9)
    ,
    OR( //(10)
        Output(Input(2), pos).hasAlert
        ,OR(
            isError(Output(Input(2), pos)),
            userCanRetrieveContent(action.getUser(), Output(Input(2),pos)))
        );//end-IMPLIES
    }//end-for
} //end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp{

/**
 * CWE_definition:
 *
 * The software constructs all or part of an LDAP query using externally-influenced input from
 * an upstream component, but it does not neutralize or incorrectly neutralizes special elements
 * that could modify the intended LDAP query when it is sent to a downstream component.
 *
 * Idea:
 * We try to inject some LDAP query strings to all input entries of the actions, then
 * we will check whether the system can neutralize the injected inputs or not.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 *     clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Second for loop iterates over all parameters of the action that contains form input.
 * (4) Stores the position of the action parameter in a variable.
 * (5) Checks the user will submit the form.
 * (6) Verifies that we did not test the URL before.
 * (7) Checks that the action does not originally contain any alert.
 * (8) Filters out the web pages with dynamic content.
 * (9) Creates the follow-up input
 * (10) Sets the LDAP query strings to the parameter of the action
 * (11) Checks that the result of the follow-up input is different than the source input
 *      Or The output contains an alert or an error page is shown
 *      Or the follow-up input can retrieve a content that has the right to access
 *
 */

MR CWE_90b {
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (3)
            var pos = action.getPosition(); // (4)
            IMPLIES(
                action.isClickOnButton && // (5)
                notTried( LDAPInjectionString() + x + action.url, Input(1).actions()
                    .get(pos).getElementURL() ) && // (6)
                !Output(Input(1), pos).hasAlert && // (7)
                userCanRetrieveContent(action.getUser(), Output(Input(1), pos)) && // (8)
                CREATE(Input(2), Input(1)) && // (9)
                Input(2).actions.get(pos).setFormInput(x, LDAPInjectionString()) // (10)
            ),
            OR(
                Output(Input(2), pos).hasAlert ,
                OR(
                    userCanRetrieveContent(action.getUser(), Output(Input(2), pos)) ,

```

```
        isError(Output(Input(2), pos)))  
    );//end-IMPLIES  
} //end-for  
} //end-for  
}  
} //end-MR  
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp{

/**
 * CWE_definition:
 *
 * The software constructs all or part of an LDAP query using externally-influenced input from
 * an upstream component, but it does not neutralize or incorrectly neutralizes special elements
 * that could modify the intended LDAP query when it is sent to a downstream component.
 *
 * Idea:
 * We try to inject some LDAP query strings to all input entries of the actions, then
 * we will check whether the system can neutralize the injected inputs or not.
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1).
 * (3) Second for loop iterates over all parameters of the action that contains a login form
 * input.
 * (4) Stores the position of the action parameter in a variable.
 * (5) Checks the user will submit the form.
 * (6) Verifies that we did not test the URL before.
 * (7) Checks that the action does not originally contain any error.
 * (8) Filters out the web pages with dynamic content.
 * (9) Creates the follow-up input
 * (10) Sets the LDAP query strings to the parameter of the action
 * (11) Checks that the result of the follow-up input is different than the source input
 * Or in the output an error is shown
 * Or the follow-up input can retrieve a content that has the right to access
 */

MR CWE_90c {
{
    keepDialogsOpen = true; // (1)
    for ( Action action : Input(1).actions()) { // (2)
        for (var x = 0; action.containLoginFormInput() && x < action.formInputs.size; x++){ // (3)
            var pos = action.getPosition(); // (4)
            IMPLIES(
                action.isClickOnButton && // (5)
                notTried(LDAPInjectionString() + x +action.url, Input(1).actions().get(pos)
                    .getElementURL()) && // (6)
                !isError(Output(Input(1),pos)) && // (7)
                userCanRetrieveContent(action.getUser(), Output(Input(1),pos)) && // (8)
                CREATE( Input(2), Input(1)) && // (9)
                Input(2).actions.get(pos).setFormInput(x, LDAPInjectionString()) // (10)
            ),
            OR(
                isError(Output(Input(2), pos)),
                userCanRetrieveContent(action.getUser(), Output(Input(2),pos))
            ); //end-IMPLIES
        } //end-for
    }
}

```

```
    }//end-for  
  }  
} //end-MR  
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp{

/**
 * CWE Definition:
 * CWE_923:
 * The software establishes a communication channel to (or from) an endpoint for privileged or
 * protected operations, but it does not properly ensure that it is communicating with the
 * correct endpoint.
 *
 * Idea:
 * The idea is to remove the certificate from user profile and try to access the webpages. If the
 * system does not have any mechanism to verifies that it is communicating with the correct
 * endpoint, it would show the content to the user while she/he is requesting for an authorized
 * action.
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Stores the parameters of the current action in a variable.
 * (3) Filters the actions that require certificate.
 * (4) Checks if the URL has not seen before, to speed up the process.
 * (5) It checks that the action is only available after being logged-in.
 * (6) Filters out the pages with dynamic content.
 * (7) Creates the follow-up input.
 * (8) Remove the certificate from the follow up input.
 * (9) Checks if the output generated by the action is different in the two cases.
 *     Or the user can retrieve the content that has right to access it, or she
 *         will see an error page.
 */

MR CWE_923 {
{
  for ( Action action : Input(1).actions() ){                                //(1)
    var pos = action.getPosition();                                           //(2)
    IMPLIES(
      getSysConfig().isBrowserCertificateRequired() &&                       //(3)
      notTried(action.url, Input(1).actions().get(pos).getElementURL() ) &&  //(4)
      afterLogin(action) &&                                                    //(5)
      userCanRetrieveContent(action.getUser(), Output(Input(1),pos))&&        //(6)
      CREATE ( Input(2), Input(1) ) &&                                         //(7)
      Input(2).setUseRemovedCertificate(true)                                 //(8)
    ,
    OR(                                                                         //(9)
      different( Output(Input(1),pos), Output(Input(2),pos)),
      isError(Output(Input(2),pos)))
    );
  }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * The software uses CRLF (carriage return line feeds) as a special element, e.g. to separate
 * lines or records, but it does not neutralize or incorrectly neutralizes CRLF sequences from
 * inputs.
 *
 *
 * Idea:
 * CRLF injections are vulnerabilities where the attacker is able to inject CR (carriage return,
 * ASCII 13) and LF (line feed, ASCII 10) characters into the web application. This lets the
 * attacker add extra headers to HTTP responses or even make the browser ignore the original
 * content and process injected content instead.
 * Then, we inject some CRLF strings e.g. %0d%0a that %0d and %0a are encoded forms of \r and \n
 * respectively, to the URL of the action to check whether the system can neutralize the input or
 * not.
 * We expect the system to ignore the injected CRLF strings or shows an error page.
 *
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Defines a variable to store the position of the Input(1)'s action.
 * (3) Creates a new URL which includes CRLF strings.
 * (4) Checks that the URL has not been previously seen.
 * (5) Filters out the web pages with dynamic content.
 * (6) Creates the follow-up input by copying the Input(1).
 * (7) Sets the modified URL to the follow-up input.
 * (8) The user may retrieve an empty file or see an alert
 *     Or the user may see a content that already has right to access it
 *     Or the system may show an error page to the user.
 */

MR CWE_93a {
{
    for ( Action action : Input(1).actions() ) {                                //(1)
        var pos = action.getPosition();                                           //(2)
        var newUrl = action.url + CRLFAttackString();                           //(3)
        IMPLIES(
            notTried( action.getUser(), newUrl )                                &&                               //(4)
            userCanRetrieveContent(action.getUser(), Output(Input(1),pos))&&      //(5)
            CREATE( Input(2), Input(1) ) &&                                       //(6)
            Input(2).actions().get(pos).setUrl( newUrl )                        //(7)
        ,
        OR(                                                                        //(8)
            OR(
                Output(Input(2), pos).emptyFile,
                Output(Input(2), pos).hasAlert),
            OR(
                userCanRetrieveContent(action.getUser(), Output(Input(2),pos)),

```

```
        isError(Output(Input(2), pos)))  
    ); //end-IMPLIES  
} //end-for  
}  
} //end-MR  
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * The software uses CRLF (carriage return line feeds) as a special element, e.g. to separate
 * lines or records, but it does not neutralize or incorrectly neutralizes CRLF sequences from
 * inputs.
 *
 *
 * Idea:
 * CRLF injections are vulnerabilities where the attacker is able to inject CR (carriage return,
 * ASCII 13) and LF (line feed, ASCII 10) characters into the web application. This lets the
 * attacker add extra headers to HTTP responses or even make the browser ignore the original
 * content and process injected content instead.
 * Then, we inject some CRLF strings e.g. %0d%0a that %0d and %0a are encoded forms of \r and \n
 * respectively, to the URL of the action to check whether the system can neutralize the input or
 * not.
 * We expect the system to ignore the injected CRLF strings or show an error page.
 * In this version we will inject the CRLF strings after a separator in URL.
 *
 *
 * Comments:
 * (1) For loop iterates over all actions of the Input(1).
 * (2) Defines a variable to store the position of the Input(1)'s action.
 * (3) Creates a new URL which includes CRLF strings.
 * (4) Checks that the URL has not been previously seen.
 * (5) Filters out the web pages with dynamic content.
 * (6) Creates the follow-up input by copying the Input(1).
 * (7) Sets the modified URL to the follow-up input.
 * (8) The user may retrieve an empty file or see an alert
 *     Or the user may see a content that already has right to access it
 *     Or the system may show an error page to the user.
 */

MR CWE_93b {
{
    var sep="/";
    for ( var par=0; par < 3; par++ ){
        for ( Action action : Input(1).actions() ) {
            var pos = action.getPosition();
            var newUrl = action.url + sep+ CRLFAttackString();
            IMPLIES(
                notTried( action.getUser(), newUrl )           &&
                userCanRetrieveContent(action.getUser(), Output(Input(1),pos))&&
                CREATE( Input(2), Input(1) ) &&
                Input(2).actions().get(pos).setUrl( newUrl )
            ,
            OR(
            OR(

```

```
        Output(Input(2), pos).emptyFile,  
        Output(Input(2), pos).hasAlert ),  
OR(  
    userCanRetrieveContent(action.getUser(), Output(Input(2),pos)) ,  
    isError(Output(Input(2), pos)))  
); //end-IMPLIES  
} //end-for  
} //end-for  
sep=sep+"../";}  
} //end-MR  
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import sun.security.util.Password

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_94: The software constructs all or part of a code segment using externally-influenced
 * input from an upstream component, but it does not neutralize or incorrectly neutralizes
 * special elements that could modify the syntax or behavior of the intended code segment.
 *
 * CWE_95 (child of CWE_94): The software receives input from an upstream component, but it does
 * not neutralize or incorrectly neutralizes code syntax before using the input in a dynamic
 * evaluation call (e.g. "eval").
 *
 * Idea:
 * We will try to inject some code as the value of the parameter and the system should not allow
 * the action to be done successfully. It should show an error page or show the content that the
 * user already has the right to access it.
 *
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 * clicking on OK.
 * (2) For loop iterates over all actions of the Input(1)
 * (3) Stores the parameters of the current action in a variable
 * (4) Iterates over the actions that contain a form input.
 * (5) Checks if the action submits the form; i.e. the action contains clicking on submit button.
 * (6) Filters the dynamic content.
 * (7) Verifies that the URL has not been tried before.
 * (8) Checks the action does not contain any alert message initially.
 * (9) Creates Input(2) as the follow-up input.
 * (10) Set the code injection string as the form input value.
 * (11) The system should show an error page or a content that the user has right to access.
 *
 */

MR CWE_94_95 {
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        var pos = action.getPosition(); // (3)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (4)
            IMPLIES(
                action.isClickOnButton && // (5)
                userCanRetrieveContent(action.getUser(), Output(Input(1),pos))&& // (6)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL()) && // (7)
                !Output(Input(1),pos).hasAlert && // (8)
                CREATE(Input(2), Input(1)) && // (9)
                Input(2).actions.get(pos).setFormInput(x, CodeInjectionString()) // (10)
            ,
            OR( // (11)
                isError(Output(Input(2), pos)),

```

```
    userCanRetrieveContent(action.getUser(), Output(Input(2),pos)))  
    ); //end-IMPLIES  
  } //end-for  
} //end-for  
}  
} //end-MR  
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import smrl.mr.language.actions.StandardAction;
import jdk.internal.util.xml.impl.Input

package smrl.mr.owasp{

/**
 *
 * CWE Definition:
 * CWE_94: The software constructs all or part of a code segment using externally-influenced
 * input from an upstream component, but it does not neutralize or incorrectly neutralizes
 * special elements that could modify the syntax or behavior of the intended code segment.
 *
 * CWE_96 (child of CWE_94): The software receives input from an upstream component, but it does
 * not neutralize or incorrectly neutralizes code syntax before inserting the input into an
 * executable resource, such as a library, configuration file, or template.
 *
 * This issue is most frequently found in PHP applications that allow users to set configuration
 * variables that are stored within executable PHP files.
 *
 * Idea:
 * While the programmer intends for the input sections i.e. messages; to only include data, an
 * attacker can provide a message such as:
 *         message=%3C?php%20system(%22/bin/ls%20-l%22);?%3E
 * That can be decoded to
 *         <?php system("/bin/ls -l");?>
 * Then in this MR we are trying to inject some malformed strings(like the above) to the all
 * input entries.
 * The system should not allow to complete the task by showing an error page or showing the
 * content that the user has right to access it.
 *
 *
 *
 * Comments:
 * (1) It avoids clicking on OK; because dialogs are normally ignored by our framework by
 *     clicking on OK.
 * (2) For loop iterate over all actions of the Input(1)
 * (3) Define a variable to store the position of the Input(1)'s action
 * (4) The second loop iterates over all parameters of the action if the action contains a form
 *     input.
 * (5) makes sure the user will submit the form.
 * (6) Verifies the current parameter was not tested before.
 * (7) Filters out the web pages with dynamic content.
 * (8) Creates the follow-up input by copying the Input(1).
 * (9) Injects some Static injection strings to the follow-up input.
 * (10) The system may show an error page to the user
 *      Or the user is retrieving the content that has right to access it.
 */
}

```

```

MR CWE_94_96_B {
{
    keepDialogsOpen = true; // (1)
    for (Action action : Input(1).actions()){ // (2)
        var pos = action.getPosition(); // (3)
        for (var x = 0; action.containFormInput() && x < action.formInputs.size; x++){ // (4)
            IMPLIES(
                action.isClickOnButton && // (5)
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL() ) && // (6)
                userCanRetrieveContent(action.getUser(), Output(Input(1),pos)) && // (7)
                CREATE(Input(2), Input(1)) && // (8)
                Input(2).actions.get(pos).setFormInput(x, StaticInjectionString()) // (9)
            ,
            OR( // (10)
                isError(Output(Input(2), pos)),
                userCanRetrieveContent(action.getUser(), Output(Input(2),pos)))
            ); // end-IMPLIES
        } // end-for parameter
    } // end-for action
}
} // end-MR
} // end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
import sun.security.util.Password

package smrl.mr.owasp {

/**
 *
 * CWE Definition:
 * CWE_94: The software constructs all or part of a code segment using externally-influenced
 * input from an upstream component, but it does not neutralize or incorrectly neutralizes
 * special elements that could modify the syntax or behavior of the intended code segment.
 *
 * CWE_96 (child of CWE_94): The software receives input from an upstream component, but it does
 * not neutralize or incorrectly neutralizes code syntax before inserting the input into an
 * executable resource, such as a library, configuration file, or template.
 *
 * Idea:
 * While the programmer intends for the input sections i.e. messages; to only include data, an
 * attacker can provide a message such as:
 *         message=%3C?php%20system(%22/bin/ls%20-l%22);?%3E
 * That can be decoded to
 *         <?php system("/bin/ls -l");?>
 * Then in this MR we are trying to inject some malformed strings(like the above) to the all
 * input entries.
 * The system should not allow to complete the task by showing an error page or showing the
 * content that the user has right to access it.
 *
 * Comments:
 * (1) For loop iterate over all actions of the Input(1)
 * (2) Stores the position of the action in a variable.
 * (3) The second loop iterates over all parameters of the action.
 * (4) Verifies the current parameter was not tested before.
 * (5) Filters out the web pages with dynamic content.
 * (6) Creates the follow-up input by copying the Input(1).
 * (7) Injects some Static injection strings to the follow-up input.
 * (8) The system may show an error page to the user
 *     Or the user is retrieving the content that has right to access it.
 */

MR CWE_94_96 {
{
    for ( Action action : Input(1).actions() ) {                                //(1)
        var pos = action.getPosition();                                           //(2)
        for ( var x =0; x < action.parameters.size; x++){                       //(3)
            IMPLIES(
                notTried( x+action.url, Input(1).actions().get(pos).getElementURL() ) &&           //(4)
                userCanRetrieveContent(action.getUser(), Output(Input(1),pos)) &&           //(5)
                CREATE ( Input(2), Input(1) ) &&                                   //(6)
                Input(2).actions().get(pos).setParameterValue( x, StaticInjectionString() )   //(7)
            ,
            OR(                                                                    //(8)
                isError(Output(Input(2), pos)),
                userCanRetrieveContent(action.getUser(), Output(Input(2),pos)))
            )
        }
    }
}
}

```

```
        );//end-IMPLIES
    }//end-for
} //end-for
}
} //end-MR
} //end-package
```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
package smrl.mr.owasp{

/*
 * Definition:
 * If an action is not expected to be executed by a user 'u' (because is not
 * available in his GUI), then user 'u' should not be able to execute that action
 * even if he changes the user id parameter in that action (i.e., the action
 * should lead to different results when executed by a valid and invalid user).
 *
 * Comments:
 * (1) The first loop iterates over all the actions of the input sequence.
 * (2) The second iterates over all the parameters of the action to identify a parameter that
 * specifies the user id.
 * (3) Stores the position of the current action in a variable.
 * (4) Checks if the current action contains a user ID.
 * (5) Defines the follow-up input as a copy of the source input.
 * (6) Changes the user ID to the one of User.
 * (7) Changes the login credentials of the follow-up input.
 * (8) Checks if the output generated by the action is different in the two cases.
 */
MR OTG_AUTHZ_003 {
{
  for ( Action action : Input(1).actions() ){                                //(1)
    for (var par=0; par < action.getParameters().size(); par++){              //(2)
      var pos = action.getPosition();                                          //(3)
      IMPLIES (
        isUserIdParameter(action,par,action.getUser() ) &&                  //(4)
        (CREATE ( Input(2), Input(1) ) &&                                     //(5)
        Input(2).actions().get(pos).setParameterValue(par,User()) ) &&      //(6)
        CREATE (Input(3), changeCredentials(Input(1), User()) )              //(7)
        ,
        different (Output(Input(2),pos), Output(Input(3),pos))                //(8)
      );//end-IMPLIES
    }//end-for
  }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
package smrl.mr.owasp{

/*
 * Definition:
 * Some URLs are expected to be used only once. These URLs can be identified
 * (by the data collection framework) by checking if a same action (e.g., clicking on a button)
 * triggers always different (e.g., the button URL is always different) over different
 * executions.
 * In this case a user should not be able to reuse the URL multiple times (e.g., sending POST
 * data to the same URL).
 *
 * Comments:
 * (1) The loop iterates over all the actions of the input.
 * (2) Stores the position of the current action in a variable.
 * (3) Checks if the URL of the current action changes over multiple executions.
 * (4) Defines the follow-up input as a copy of the source input where the action above is
 *     duplicated.
 * (5) Checks if the output generated by the second action different than in the case of the
 *     first action.
 */

MR OTG_BUSLOGIC_005 {
{
    for ( Action action : Input(1).actions() ){                //(1)
        var pos = action.getPosition();                          //(2)
        IMPLIES( (
            urlOfActionChangesOverMultipleExecutions( action ) &&    //(3)
            CREATE ( Input(2), addAction( Input(1), pos, action ))    //(4)
            ,
            different( Output(Input(1),pos), Output(Input(2), pos) )    //(5)
        );//end-IMPLIES
        }//end-for
    }
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
package smrl.mr.owasp {

/*
 * Definition:
 * This MR checks that actions available with one HTTP method (e.g., POST )
 * should not be available with another method (e.g., DELETE).
 *
 * Comments:
 * (1) Iterates over all the actions of an input sequence.
 * (2) Stores the position of the current action in a variable.
 * (3) Verifies the URL has not tried before.
 * (4) Defines a follow-up input.
 * (5) Sets a different HTTP method to perform the selected action.
 * (6) verifies that the output generated by the modified action is different in the two cases.
 */

MR OTG_INPVAL_003 {
{
  for ( Action action : Input(1).actions() ) {                                     //(1)
    var pos = action.getPosition();                                              //(2)
    IMPLIES(
      notTried( HttpMethod(), Input(1).actions().get(pos).getUrl() ) &&        //(3)
      CREATE( Input(2) , Input(1) ) &&                                          //(4)
      Input(2).actions().get(pos).setMethod( HttpMethod() )                   //(5)
      ,
      different ( Output(Input(1),pos), Output(Input(2),pos) )                 //(6)
    );//end-IMPLIES
  }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
package smrl.mr.owasp{

/*
 * Definition:
 * Duplicating a parameter value should not lead to a different behaviour in the system.
 *
 * Comments:
 * (1) The first loop iterates over all the actions of an input sequence.
 * (2) The second loop iterates over all the parameters.
 * (3) Stores the position of the current action in a variable.
 * (4) Defines a follow-up input.
 * (5) Duplicates one parameter.
 * (6) verifies that the output generated by the modified action is the same in the two cases.
 */
MR OTG_INPVAL_004 {
{
    for ( Action action : Input(1).actions() ){                //(1)
        for (var par=0; par < action.getParameters().size(); par++ ){                //(2)
            var pos = action.getPosition();                //(3)
            IMPLIES (
                (CREATE ( Input(2), Input(1)) &&                //(4)
                Input(2).actions().get(pos).addParameter(                //(5)
                    action.getParameterName(par),
                    action.getParameterValue(par)))

                ,
                equal ( Output(Input(1) ) , Output( Input(2)))                //(6)
            );//end-IMPLIES
        }//end-for
    }//end-for
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
package smrl.mr.owasp{

/*
 * Definition:
 * A login action performed by a user already authenticated should always trigger the generation
 * of a new session ID.
 *
 * Comments:
 * (1) The first loop iterates over the inputs to find a sign up action
 * (2) The second loop iterates over the actions that follow the sign up. The second loop is
 *     necessary to check that a sign up action repeated at any point of the action sequence
 *     leads to a new session ID.
 * (3) Checks if the current action has been performed after a login.
 * (4) Defines a follow-up input with the sign up action being duplicated in a certain position.
 * (5) Checks if the session ID of the response page sent after the two successive login actions
 *     is different.
 *
 */

MR OTG_SESS_003 {
{
    for( Action signup : Input(1).actions() ){                //(1)
        for ( var i=0;isSignup(signup) && i < Input (2).actions().size; i++ ) {                //(2)
            var f = Input (2).actions().get(i);
            var pos = f.getPosition();
            IMPLIES (
                afterLogin ( f ) &&                                //(3)
                CREATE (Input (3), addAction ( Input(2), pos+1, signup ))                //(4)
                ,
                different(                                         //(5)
                    Output(Input(3), pos).getSession(),
                    Output(Input(3), pos+1).getSession())
                );//end-IMPLIES
            }//end-for
        }//end-for
    }
}
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
package smrl.mr.owasp{

/*
 * Definition:
 * After a session timeout the user should not be able to perform an action that requires to be
 * logged in.
 *
 * Comments:
 * (1) The loop iterates over all the actions to find actions executed within a session, after
 *     login.
 * (2) Checks that the action is generally not available without login.
 * (3) Checks if the session is not null.
 * (4) Checks that a session timeout is set.
 * (5) Defines a follow-up input where the selected action is executed after timeout (usually
 *     simulated).
 * (6) Checks if the output of the action generated after timeout is different than in the case
 *     in which it is executed before the timeout.
 */
MR OTG_SESS_007 {
{
  for ( Action action : Input(1).actions() ){                                //(1)
    IMPLIES (
      notAvailableWithoutLoggingIn(action) &&                               //(2)
      NOT ( NULL ( action.session ) ) &&                                     //(3)
      action.session.timeout > 0 &&                                           //(4)
      CREATE ( Input(2) , addAction( Input( 1 ), action.position,           //(5)
        Wait(action.session.timeout)))
      ,
      different (                                                            //(6)
        Output( Input(1), action.position ),
        Output( Input(2), action.position ) )
      );//end-IMPLIES
    }//end-for
  }
} //end-MR
} //end-package

```

```

import static smrl.mr.language.Operations.*;
import smrl.mr.language.Action;
package smrl.mr.owasp{

/*
 * Definition:
 * An action that (1) is available without logging in and (2) generates a session,
 * should not enable a user to execute an action that requires to be logged in.
 *
 * Comments:
 * (1) Stores the action that is available without a login in a variable.
 * (2) The first loop iterates over all the actions of the input.
 * (3) Checks that the current action is not available without being logged in.
 * (4) Looks for an action available without being logged-in that generates a session.
 * (5) Defines a follow-up input that executes two actions, the action available without
 *     being logged in, and the selected action (i.e., the one available only by being logged-
 *     in).
 * (6) Checks that the output of the action is different when execute with and without being
 *     logged in (even if after an action that does not require a log-in but generates a
 *     session).
 */

MR OTG_SESS_008 {
{
    var ActionAvailableWithoutLogin = ActionAvailableWithoutLogin();           //(1)
    for ( Action action : Input(1).actions() ){                               //(2)
        IMPLIES(
            notAvailableWithoutLoggingIn( action ) &&                         //(3)
            NOT (NULL(ActionAvailableWithoutLogin.getSession())) &&           //(4)
            CREATE( Input(2) , Input( ActionAvailableWithoutLogin, action))    //(5)
        ,
        different(                                                              //(6)
            Output( Input(1), action.position ),
            Output( Input(2), 1 ) )
        );//end-IMPLIES
    }//end-for
}
} //end-MR
} //end-package

```